GROUP ASSIGNMENT

CT107-3-3-TXSA

TEXT ANALYTICS AND SENTIMENT ANALYSIS

CSSE___CT107-3-3-TXSA-L-2___2025-05-23

HAND OUT DATE: 30 MAY 2025

HAND IN DATE: 29 AUGUST 2025

WEIGHTAGE: 50%

INSTRUCTIONS TO CANDIDATES:

1 **Submit your assignment via Moodle.**

2 **Students are advised to underpin their answers with the use of references (cited using the 7th Edition of APA Referencing Style).**

3 **Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.**

4 **Cases of plagiarism will be penalized.**

5 **You must obtain 50% overall to pass this module.**

Name	TP Number
Yehia Rasheed Yehia Al Haddad	TP072510
Ebrahim Shaker	TP065022
Mohammadomar Hashemi	TP074353
Muhammad Zaman	TP072946

Part A 6
Q1: Tokenization and Filter stop words & punctuation 6
Q2: Word Stemming 10
Q3: Form Parts of Speech (POS) taggers & Syntactic Analysers 16
Q4: Work on Sentence Probabilities 22
Q5: Individual Component 28
Ebrahim Shaker 28
Yehia Rasheed 31
MohammedOmar Hashemi 34
Muhammad Zaman 38
Part B 42
Q1: Dataset and Exploratory Data Analysis (EDA) 42
1.0 Dataset Choice, Classification Problem, and Suitability 42
1.1 Dataset Choice and Description: IMDB Movie Reviews 42
1.2 Classification Problem Definition 42
1.3 Suitability of the Dataset 42
1.4 Literature Review on the Dataset 43
1.5 Justification of the Dataset 45
2.0 Exploratory Data Analysis (EDA) 46
2.1 Suggested Predictive Models 66
2.2 Critical Analysis Summary 66
Q2: Supervised Text Classification 68
1.0 Build Supervised Text Classification models 68
1.1 Data Preparation & Train/Test Split 68
1.2 Logistic Regression 69
1.3 Support Vector Machine (SVM) 72
1.4 Random Forest 74
1.5 Bi-LSTM 76

2.0 Select Suitable Evaluation Measures 81
2.1 Accuracy 81
2.2 Precision 81
2.3 Recall 81
2.4 F1-Score 81
2.5 Confusion Matrix 82
3.0 Perform Model Evaluations and Present the Results 83
3.1 Logistic Regression (Best Performance) 83
3.2 Support Vector Machine (SVM) 84
3.3 Random Forest 84
3.4 Bi-LSTM (Deep Learning Approach) 85
3.5 Comparative Results Table & Visualization 86
Q3: Hyperparameter Tuning (Sentiment Classification) 89
3.1 Objective 89
3.2 Methods (abridged; full code in the submitted notebook) 89
3.3 Results (cross-validated) 96
3.4 Final evaluation (hold-out test) 97
3.5 Limitations and next steps 98
Q4 Discussion and Conclusion 98
4.1 Selected Evaluation Metrics 99
4.2 Model Evaluation and Results Presentation 99
4.2.1 Logistic Regression 99
4.2.2 Support Vector Machine (SVM) 100
4.2.3 Random Forest 101
4.2.4 Bi-LSTM 101
4.3 Overall Evaluation Metrics Comparison 102
4.4 Critical Analysis and Discussion 103
References 104

Part A

Q1: Tokenization and Filter stop words & punctuation

Word Tokenisation

Tokenisation is the act of transforming raw text into smaller pieces of data (called tokens), usually comprised of either words or punctuation. Tokenisation is the first step of text analytics because computers are not capable of understanding sentences directly. Instead, a computer needs the text broken down into units that can be counted, compared and analysed.

In this task, I applied three different methods to tokenise the text corpus:

1. **Using Python's split() function** – This separates text based only on spaces, so punctuation remains attached to words.
2. **Using Regular Expressions (Regex)** – This allows extraction of only the words by removing punctuation, though sometimes it can break complex words.
3. **Using NLTK's word_tokenize** – A more advanced, language-aware method that separates punctuation into its own tokens while preserving meaningful structures like hyphenated words.

```
# 1) split(): splits on whitespace only; punctuation stays attached
tokens_split = text.split()
print("===== Tokenization using split() =====")
print(tokens_split[:30])
print("Total tokens (split):", len(tokens_split), "\n")

# 2) Regex: extract only word characters (letters/digits/_); punctuation is dropped
tokens_regex = re.findall(r"\b\w+\b", text)
print("===== Tokenization using Regex =====")
print(tokens_regex[:30])
print("Total tokens (Regex):", len(tokens_regex), "\n")

# 3) NLTK: language-aware tokeniser; separates punctuation into its own tokens,
tokens_nltk = word_tokenize(text)
print("===== Tokenization using NLTK word_tokenize =====")
print(tokens_nltk[:30])
print("Total tokens (NLTK):", len(tokens_nltk))
```

Figure 1: Python code for word tokenisation using split(), Regex, and NLTK.

```
===== Tokenization using split() =====
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given',
Total tokens (split): 82

===== Tokenization using Regex =====
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given',
Total tokens (Regex): 83

===== Tokenization using NLTK word_tokenize =====
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given',
Total tokens (NLTK): 94
```

Figure 2: Sample outputs for tokenisation using different methods.

===== Tokenization using split() =====

['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input.', 'In', 'basic', 'classification', 'tasks,', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs,', 'and', 'the']

Total tokens (split): **82**

===== Tokenization using Regex =====

['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', 'In', 'basic', 'classification', 'tasks', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', 'and', 'the']

Total tokens (Regex): **83**

===== Tokenization using NLTK word_tokenize =====

['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification', 'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs']

Total tokens (NLTK): **94**

As we can see from these outputs, the differences are quite clear. The split() function retains the punctuation attached to words (e.g. tasks,), regex removes punctuation completely, and NLTK provides the best and most flexible option, providing punctuation as separate tokens.

Justification of the Most Suitable Method

When comparing the three methods, the strengths and weaknesses become clear:

- The **split() method** is too simplistic. It is quick but inaccurate because punctuation is left inside tokens, which can lead to errors in later analysis.
- The **Regex approach** is better because it cleans punctuation, but it is still crude. For example, it breaks *open-class* into open and class, losing meaning.
- The **NLTK word_tokenize method** is the most advanced and suitable for text analytics. It separates punctuation into its own tokens, preserves complex words, and is designed for natural language processing tasks.

Therefore, the most suitable tokenisation method is NLTK's word_tokenize. This method provides flexibility and accuracy, making it the best choice for text analytics.

Stop Words & Punctuation Removal

Even following tokenisation, the text will have a lot of words like the, is, in, and, without lots of meaningful relevance to analysis. These are referred to as stop words. Similarly, punctuation symbols such as,.;? also do not bear semantic meaning in most analytics applications. They can be removed from the text to clean it up, allowing for better analysis.

In this task, I used:

- NLTK's built-in **stop word list** (English).
- Python's string.punctuation to remove punctuation.
- I also decided to **keep negation words** like *not, no, nor* because they are important in sentiment analysis (e.g., "not good" ≠ "good").

```python
import string
from collections import Counter
from nltk.corpus import stopwords

# 1) Build stopword set (English). For sentiment tasks, it's common to KEEP negations.
stop_words = set(stopwords.words("english"))
```

Figure 3:Python code for removing stop words and punctuation.

```
===== Stopwords found in the corpus (unique) =====
['a', 'all', 'and', 'are', 'be', 'each', 'for', 'from', 'has', 'in', 'is', 'of', 'other', 'the']

===== Stopword frequencies in the corpus =====
Counter({'in': 7, 'the': 5, 'of': 5, 'is': 4, 'a': 3, 'for': 2, 'each': 2, 'and': 2, 'from': 1, 'all': 1, 'other': 1, 'has': 1, 'be': 1, 'are': 1})

===== Cleaned tokens (no stopwords, no punctuation) — preview =====
['Classification', 'task', 'choosing', 'correct', 'class', 'label', 'given', 'input', 'basic', 'classification', 'tasks', 'input', 'considered', 'is
Total cleaned tokens: 46
```

Figure 4: Output showing stop words identified, their frequencies, and the cleaned tokens list.

```python
# 3) Lowercase a view of tokens for stopword matching (case-insensitive)
l

# 4) Find which stopwords actually appear in the corpus (for reporting)
stopwords_in_text = [t for t in lower_tokens if t in stop_words_adjusted]
unique_stopwords_found = sorted(set(stopwords_in_text))
stopword_freq = Counter(stopwords_in_text)

# 5) Remove punctuation tokens (they are separate tokens in NLTK)
def is_punctuation(tok: str) -> bool:
    return (tok in punct) or all(ch in punct for ch in tok)

tokens_no_punct = [t for t in tokens_nltk if not is_punctuation(t)]

# 6) Remove stopwords (case-insensitive), but preserve negations if configured
clean_tokens = [t for t in tokens_no_punct if t.lower() not in stop_words_adjusted]

print("===== Stopwords found in the corpus (unique) =====")
print(unique_stopwords_found)
print("\n===== Stopword frequencies in the corpus =====")
print(stopword_freq)
print("\n===== Cleaned tokens (no stopwords, no punctuation) — preview =====")
print(clean_tokens[:40])
print("Total cleaned tokens:", len(clean_tokens))
```

===== Stopwords found in the corpus (unique) =====

['a', 'all', 'and', 'are', 'be', 'each', 'for', 'from', 'has', 'in', 'is', 'of', 'other', 'the']

===== Stopword frequencies in the corpus =====

Counter({'in': 7, 'the': 5, 'of': 5, 'is': 4, 'a': 3, 'for': 2, 'each': 2, 'and': 2, 'from': 1, 'all': 1, 'other': 1, 'has': 1, 'be': 1, 'are': 1})

===== Cleaned tokens (no stopwords, no punctuation) — preview =====

['Classification', 'task', 'choosing', 'correct', 'class', 'label', 'given', 'input', 'basic', 'classification', 'tasks', 'input', 'considered', 'isolation', 'inputs', 'set', 'labels', 'defined', 'advance', 'basic',

'classification', 'task', 'number', 'interesting', 'variants', 'example', 'multiclass', 'classification', 'instance', 'may', 'assigned', 'multiple', 'labels', 'open-class', 'classification', 'set', 'labels', 'not', 'defined', 'advance']

Total cleaned tokens: 46

Importance of Filtering Stop Words & Punctuation

Filtering stop words and punctuation is a crucial step in text analytics for several reasons:

1. Eliminates noise: Stop words are used very often but don't have useful information; their removal allows us to eliminate unnecessary noise in our dataset.
2. Improves accuracy: With only meaningful tokens present, the models and algorithms cannot be distracted by filler that is not directly related to the main subject.
3. Reduces dimensionality: Reducing the size of the vocabulary generally makes them faster and requires less memory.
4. Cleans up sentiment: When stopwords were removed (with the exception of negations) there were words remaining that were not complicating the process of sentiment polarity.
5. Easier visualisation: More meaningful word clouds, frequency plots or keyword extractions can be created from cleaned data.

In short, filtering stop words and punctuation makes the text cleaner, more efficient, and more meaningful for further analysis.

Q2: Word Stemming

Word Stemming

Stemming is a vital process in Natural Language Processing (NLP) that shortens words to their base form or root form. Stemming allows for using various word forms (e.g., running, runner, ran) within the same word category, while not over-complicating, or enriching available data that models can use to learn.

In the context of text analytics, stemming helps reduce the dimensionality of text in the following ways:

- **Dimensionality Reduction:** When different variations of the word are transformed to a common root form, stemming reduces the ratio of unique words in the text dataset. This alleviates redundancy and allows for a model to concentrate on similar essential features in the text.

- **Model Efficiency:** Meaningful words with overlapping meanings (i.e., run, running, and ran) can be reduced to a common word form. These common forms allow models to built some ground-up familiarity with how to establish meaning for overlapping words during use, improving the models accuracy and generalizability.

- **Model Performance:** Stemming removes much of the background noise that makes it difficult to analyze, and condenses the messy details into more manageable forms. Removing noise makes the model more flexible and frees up further space for identifying some attributes to build actionable patterns during its train process.
-

Methods for Word Stemming

In this demonstration of stemming, we will use three different methods on the following sentence:

" Classification is the task of choosing the correct class label for a given input. In basic classification tasks, each input is considered in isolation from all other inputs, and the set of labels is defined in advance. The basic classification task has a number of interesting variants. For example, in multiclass classification, each instance may be assigned multiple labels; in open-class classification, the set of labels is not defined in advance; and in sequence classification, a list of inputs are jointly classified. "

1. Regular Expression (Regex) Stemming:

Regular Expression stemming is a straightforward approach where suffixes like "ing", "ly", and "ed" are removed from words. This method is simple to accomplish and fast, but does not handle complicated issues or those which have an irregular form

Here is the implementation of Regex stemming:

```python
import re

# Data_1.txt text
sentence = """Classification is the task of choosing the correct class label for a given input.
In basic classification tasks, each input is considered in isolation from all other inputs,
and the set of labels is defined in advance. The basic classification task has a number of interesting variants.
For example, in multiclass classification, each instance may be assigned multiple labels;
in open-class classification, the set of labels is not defined in advance;
and in sequence classification, a list of inputs are jointly classified."""

# Tokenize
words = sentence.lower().split()

# Regex stemmer
def regex_stem(word):
    return re.sub(r'(ing|ly|es|ed)$', '', word)

regex_stems = [regex_stem(w) for w in words]

print("Original words (first 20):", words[:20])
print("Regex Stems (first 20):", regex_stems[:20])
```

Figure 5: Regular Expression (Regex) Stemming code

Output:

Original words (first 20): ['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input.', 'in', 'basic', 'classification', 'tasks,', 'each', 'input']

Regex Stems (first 20): ['classification', 'is', 'the', 'task', 'of', 'choos', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input.', 'in', 'basic', 'classification', 'tasks,', 'each', 'input']

The method will strip off the common suffixes, reducing words such as “defined → defin”, “classification → classif”.

2. Porter Stemmer (NLTK):

The Porter Stemmer is one of the most popular stemming algorithms. It uses a set of rules to strip suffixes from words. This is a more intelligent approach than regex stemming and works well for a lot of English words. However, it also sometimes is overly aggressive, reducing words to too simplified a state.

Here's how the Porter Stemmer operates:

```python
import nltk
from nltk.stem import PorterStemmer

# Make sure punkt is available (for other NLTK tasks if needed)
nltk.download('punkt')

# Data_1.txt text
sentence = """Classification is the task of choosing the correct class label for a given input.
In basic classification tasks, each input is considered in isolation from all other inputs,
and the set of labels is defined in advance. The basic classification task has a number of interesting variants.
For example, in multiclass classification, each instance may be assigned multiple labels;
in open-class classification, the set of labels is not defined in advance;
and in sequence classification, a list of inputs are jointly classified."""

# Tokenize
words = sentence.lower().split()

# Porter Stemmer
porter = PorterStemmer()
porter_stems = [porter.stem(w) for w in words]

print("Original words (first 20):", words[:20])
print("Porter Stems (first 20):", porter_stems[:20])
```

Figure 6: Porter Stemmer (NLTK) code

Output:

Original words (first 20): ['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input.', 'in', 'basic', 'classification', 'tasks,', 'each', 'input']

Porter Stems (first 20): ['classif', 'is', 'the', 'task', 'of', 'choos', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input.', 'in', 'basic', 'classif', 'tasks,', 'each', 'input']

The Porter Stemmer acts similar to regex, reducing “considered” to “consid”, “classification to “classif”; but the Porter Stemmer can take more liberties with some other words, like turning "happily" into "happili".

3. Lancaster Stemmer (NLTK):

The Lancaster Stemmer is a more aggressive stemming algorithm than the Porter Stemmer. It usually deletes more characters from words, which sometimes leads to a process called over-

stemming which makes the stems less meaningful by deleting everything except for perhaps two characters. This method is suited for applications that prescribe more extreme cuts of word forms but can be more lenient on the accuracy of the stems.

The Lancaster stemmer works as follows:

```python
import nltk
from nltk.stem import LancasterStemmer

# Make sure punkt is available (for other NLTK tasks if needed)
nltk.download('punkt')

# Data_1.txt text
sentence = """Classification is the task of choosing the correct class label for a given input.
In basic classification tasks, each input is considered in isolation from all other inputs,
and the set of labels is defined in advance. The basic classification task has a number of interesting variants.
For example, in multiclass classification, each instance may be assigned multiple labels;
in open-class classification, the set of labels is not defined in advance;
and in sequence classification, a list of inputs are jointly classified."""

# Tokenize
words = sentence.lower().split()

# Lancaster Stemmer
lancaster = LancasterStemmer()
lancaster_stems = [lancaster.stem(w) for w in words]

print("Original words (first 20):", words[:20])
print("Lancaster Stems (first 20):", lancaster_stems[:20])
```

Figure 7: Lancaster Stemmer (NLTK) code

Output:

Original words (first 20): ['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input.', 'in', 'basic', 'classification', 'tasks,', 'each', 'input']

Lancaster Stems (first 20): ['class', 'is', 'the', 'task', 'of', 'choos', 'the', 'correct', 'class', 'label', 'for', 'a', 'giv', 'input.', 'in', 'bas', 'class', 'tasks,', 'each', 'input']

The Lancaster Stemmer provides similar results to the Porter Stemmer, however it is much more aggressive. For example, “classification → class” or “number → numb”..

<u>Three Stemming Methods Comparision</u>

Regular Expression Stemmer:

- Advantages: Fast, easy to implement, can be tailored to specific suffixes.
- Disadvantages: Recognizes no irregular forms and has limited capabilities in some cases.
- Example: "defined" → "defin", "choosing" → "choos"

Porter Stemmer:

- Advantages: Well known, fairly accurate in a lot of cases. Reasonable tradeoff between efficiency and accuracy.
- Disadvantages: Can sometimes over-stem: "happily" → "happili".
- Example: "classification" → "classif", "considered" → "consid"

Lancaster Stemmer:

- Advantages: Aggressive, so could be useful reducing the feature space dramatically.
- Disadvantages: Tends to over-stem changing the information too much and possibly the meaning of the words.
- Example: "classification" → "class", "number" → "numb"

<u>Conclusion</u>

Every stemming technique has its own pros and cons:

- A Regex Stemmer is fast and lightweight, but lacks anykind of flexibility and sophistication
- A Porter Stemmer has a reasonable tradeoff of accuracy to simplicity, and is good for its general use
- A Lancaster Stemmer takes an aggressive approach to stemming and could produce overly-aggressively simplified stems. This approach can be beneficial in some instances, but should be used sparingly due to the risk of over-stemming.

To sum up, the specific details of the analysis will determine which stemming approach is best. If the goal is speed and cheapness, you may even get by with regex. If you need more accurate and flexible stems, then go with the Porter stemmer. If your need is for aggressive reduction, if your need is needing the Lancaster stemmer.

Q3: Form Parts of Speech (POS) taggers & Syntactic Analysers

Introduction:

In this section, Natural Language Processing (NLP) is applied to Data_2.txt. NLTK punkt (with a fallback to a regular expression) is used to pre-process the text and tokenise it. Three complementary methods are used to obtain parts-of-speech (POS): the averaged-perceptron tagger of NLTK (data-driven, context-sensitive), the TextBlob (lightweight wrapper that generally agrees with NLTK on general English), and a Regex rule-based tagger (deterministic baseline) to compare learned context with simple patterns. To analyse syntactically, a shallow chunk parser is constructed using NLTKs RegexpParser and a small grammar of NP, VP, PP, ADJP, ADVP, which gives a concise phrase-structure representation of the sentence and data to comment on tagger behaviour and limitations.

```python
from pathlib import Path
import re
import nltk
from nltk import word_tokenize, pos_tag, RegexpTagger
from textblob import TextBlob

def _ensure(pkg_name: str, resource_path: str):
    try:
        nltk.data.find(resource_path)
        return True
    except LookupError:
        try:
            print(f"[setup] downloading: {pkg_name}")
            nltk.download(pkg_name, quiet=True)
            nltk.data.find(resource_path)
            return True
        except Exception as e:
            print(f"[warn] could not get {pkg_name}: {e}")
            return False

need_punkt     = _ensure("punkt", "tokenizers/punkt")
_ensure("punkt_tab", "tokenizers/punkt_tab")
_ensure("averaged_perceptron_tagger_eng", "taggers/averaged_perceptron_tagger_eng")
_ensure("averaged_perceptron_tagger", "taggers/averaged_perceptron_tagger")

DATA_FILE = Path(r"C:\Users\User\Downloads\Part A Dataset\Part A Dataset\Data_2.txt")
text = DATA_FILE.read_text(encoding="utf-8").strip()

try:
    sentences = nltk.sent_tokenize(text)
except LookupError:
    print("[warn] punkt not available; using regex sentence split")
    sentences = re.split(r"(?<=[.!?])\s+", text)

sentence = sentences[0] if sentences else text
print("\nSelected sentence from Data_2.txt:")
print(sentence)

tokens = word_tokenize(sentence) if need_punkt else sentence.split()
```

Figure 8: *Dataset loading, sentence selection, tokenization.*

```
Selected sentence from Data_2.txt:
The big black dog barked at the white cat and chased away.
```

Figure 9: (Output) *Selected sentence (console).*

Justification:

We train the text to NLP by loading Data_2.txt, choosing the target sentence, and tokenising it. The punkt of NLTK splits sentences and words with a regular expression backup, guaranteeing reproducible preprocessing prior to POS tagging and parsing (as with the workflow in the previous sections).

```
try:
    tb = TextBlob(sentence)
    tb_tags = tb.tags
except Exception as e:
    print(f"[warn] TextBlob tagging failed ({e}); continuing without TB.")
    tb_tags = [(w, None) for w in tokens]

print("\n===== TextBlob POS tags (preview) =====")
print(tb_tags[:25])

patterns = [
    (r'^\.$', '.'), (r'^[0-9]+(,[0-9]{3})*(\.[0-9]+)?$', 'CD'),
    (r'^(The|the|A|a|An|an)$', 'DT'),
    (r'.*ing$', 'VBG'), (r'.*ed$', 'VBD'), (r'.*es$', 'VBZ'),
    (r'.*ould$', 'MD'), (r'.*ly$', 'RB'),
    (r'.*ness$','NN'), (r'.*ment$','NN'),
    (r'.*able$','JJ'), (r'.*ful$','JJ'), (r'.*ous$','JJ'), (r'.*ive$','JJ'), (r'.*ic$','JJ'),
    (r'.*', 'NN')
]
regex_tagger = RegexpTagger(patterns)
regex_tags = regex_tagger.tag(tokens)

print("\n===== Regex Tagger POS tags (preview) =====")
print(regex_tags[:25])

print("\n===== Comparison (token | NLTK | TextBlob | Regex) =====")
for i, tok in enumerate(tokens):
    nlt = nltk_tags[i][1] if i < len(nltk_tags) else None
    tbl = tb_tags[i][1]   if i < len(tb_tags)   else None
    rgx = regex_tags[i][1] if i < len(regex_tags) else None
    print(f"{tok:>12s} | {str(nlt):>4s} | {str(tbl):>4s} | {str(rgx):>4s}")
```

Figure 10: *NLTK, TextBlob, Regex taggers + head-to-head table.*

```
===== NLTK POS tags (preview) =====
[('The', 'DT'), ('big', 'JJ'), ('black', 'JJ'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'IN'), ('the', 'DT'), ('white', 'JJ'), ('cat', 'NN'), ('and', 'CC'), ('chased', 'VBD'), ('away', 'RB'), ('.', '.')]

===== TextBlob POS tags (preview) =====
[('The', 'DT'), ('big', 'JJ'), ('black', 'JJ'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'IN'), ('the', 'DT'), ('white', 'JJ'), ('cat', 'NN'), ('and', 'CC'), ('chased', 'VBD'), ('away', 'RB')]

===== Regex Tagger POS tags (preview) =====
[('The', 'DT'), ('big', 'NN'), ('black', 'NN'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'NN'), ('the', 'DT'), ('white', 'NN'), ('cat', 'NN'), ('and', 'NN'), ('chased', 'VBD'), ('away', 'NN'), ('.', '.')]
```

Figure 11: (Output) *Tagger previews.*

```
===== Comparison (token | NLTK | TextBlob | Regex) =====
        The |   DT |   DT |   DT
        big |   JJ |   JJ |   NN
      black |   JJ |   JJ |   NN
        dog |   NN |   NN |   NN
     barked |  VBD |  VBD |  VBD
         at |   IN |   IN |   NN
        the |   DT |   DT |   DT
      white |   JJ |   JJ |   NN
        cat |   NN |   NN |   NN
        and |   CC |   CC |   NN
     chased |  VBD |  VBD |  VBD
       away |   RB |   RB |   NN
          . |    . | None |    .
```

Figure 12: (Output) *Head-to-head comparison table*.

Explanation:

TextBlob and NLTK share common nouns and adjectives: big, black, white; dog, cat; VBD (barked, chased); IN (at); CC (and); RB (away). TextBlob usually leaves out the final punctuation, therefore None in the case of the period. The Regex tagger is deterministic and over-generalises unknowns to NN, thus most adjectives/function words (big, black, white, at, and, away) are mis-tagged. This illustrates the trade-off: easy rules vs. data-driven taggers which exploit context to disambiguate.

```python
chunk_grammar = r"""
  NP:   {<DT|PRP\$>?<JJ.*>*<NN.*>+}
  PP:   {<IN><NP>}
  VP:   {<MD>?<VB.*><NP|PP|ADJP|ADVP>*}
  ADJP: {<RB.*>*<JJ.*>+}
  ADVP: {<RB.*>+}
"""
chunker = nltk.RegexpParser(chunk_grammar)
chunk_tree = chunker.parse(nltk_tags)

print("\n===== Shallow parse (bracketed) =====")
print(chunk_tree.pformat(margin=120))

print("\n===== Pretty tree (console) =====")
chunk_tree.pretty_print()
```

Figure 13: *Chunk grammar (NP/VP/PP/ADJP/ADVP) + tree printing.*

```
===== Shallow parse (bracketed) =====
(S
  (NP The/DT big/JJ black/JJ dog/NN)
  (VP barked/VBD (PP at/IN (NP the/DT white/JJ cat/NN)))
  and/CC
  (VP chased/VBD)
  (ADVP away/RB)
  ./.)
```

Figure 14: (Output) *Bracketed tree (paste the whole block).*

```
===== Pretty tree (console) =====
                                          S
   _______________________________________|_____________________________________________________
  |     |          |                                   VP                          |         |
  |     |          |                        _______________|_____                   |         |
  |     |          |                       |                    PP                  |         |
  |     |          |                       |            ___________|_____            |         |
  |     |         NP                       |        |                NP             VP       ADVP
  |     |     ______|_______________       |        |      ___________|_______      |         |
and/CC ./. The/DT big/JJ black/JJ dog/NN barked/VBD at/IN the/DT     white/JJ cat/NN chased/VBD away/RB
```

Figure 15: (Output) *Pretty console tree.*

Explanation:

The phrase structure is NP The big black dog, VP barked with PP at the white cat, coordinated VP chased and ADVP away. This meets the requirement of drawing possible parse trees with an explicit phrase level analysis based on the POS tags.

Q4: Work on Sentence Probabilities

Unigrams Table

Unigram	Count
<s>	3
He	2
read	3
a	3
book	3
</s>	3
I	1
different	1
by	1
Danielle	1

Bigrams Table

Bigram	Count
(<s>, He)	2
(He, read)	2
(read, a)	3
(a, book)	3
(book, </s>)	2
(<s>, I)	1

(I, read)	1
(read, a)	3
(a, different)	1
(different, book)	1
(book, </s>)	2
(<s>, He)	2
(He, read)	2
(read, a)	3
(a, book)	3
(book, by)	1
(by, Danielle)	1
(Danielle, </s>)	1

3- Compute manually the sentences probabilities using the unsmoothed bigram model.

For the sentence : I read a book by Danielle </s>

Bigram	Count Formula	Probability
P(I | <s>)	count(<s>, I) / count(<s>) = 1/3	1/3
P(read | I)	count(I, read) / count(I) = 1/1	1
P(a | read)	count(read, a) / count(read) = 3/3 = 1	1

P(different | a)	count(a, different) / count(a) = 1/3	1/3
P(book | different)	count(different, book) / count(different) = 1/1	1
P(by | book)	count(book, by) / count(book) = 1/3	1/3
P(Danielle | by)	count(by, Danielle) / count(by) = 1/1	1
P(</s> | Danielle)	count(Danielle, </s>) / count(Danielle) = 1/1	1

Final Probability:

P(sentence) = (1/3) x (1) x (1) x (1/3) x (1) x (1)

P(sentence) = 1/27 ≈ 0.037

3- Compute manually the sentences' probabilities using the smoothed bigram model.

$$P(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + 1}{\text{Count}(w_{i-1}) + W}$$

Words (W) : <s>, He, I, read, a, book, by, different, Danielle, </s>

P(I | <s>)	(1 + 1) / (3 + 10) = 2/13	2/13
P(read | I)	(1 + 1) / (1 + 10) = 2/11	2/11
P(a | read)	(3 + 1) / (3 + 10) = 4/13	4/13
P(different | a)	(1 + 1) / (3 + 10) = 2/13	2/13
P(book | different)	(1 + 1) / (1 + 10) = 2/11	2/11

P(by | book)	(1 + 1) / (3 + 10) = 2/13	2/13
P(Danielle | by)	(1 + 1) / (1 + 10) = 2/11	2/11
P(</s> | Danielle)	(1 + 1) / (1 + 10) = 2/11	2/11

Final Probability:

P(sentence) = (2/13) x (2/11) x (4/13) x (2/13) x (2/11) x (2/13) x (2/11) x (2/11)

P(sentence) = 512 / 418161601 ≈ 1.224 × 10^{-6}

3- Implement in python using both unsmoothed and smoothed bigram language models and report the respective sentence probabilities.

```python
import nltk
from nltk import bigrams
from nltk.probability import ConditionalFreqDist, ConditionalProbDist, MLEProbDist, LaplaceProbDist

# TRAINING CORPUS
sentences_data = [
    ["<s>", "He", "read", "a", "book", "</s>"],
    ["<s>", "I", "read", "a", "different", "book", "</s>"],
    ["<s>", "He", "read", "a", "book", "by", "Danielle", "</s>"]
]

# TOKENIZE THE SENTENCES AND GENERATE BIGRAMS
all_tokens = [word for sentence in sentences_data for word in sentence]
bigram_pairs = list(bigrams(all_tokens))

# UNSMOOTHED BIGRAM PROBABILITIES
freq_dist_unsmoothed = ConditionalFreqDist(bigram_pairs)
prob_dist_unsmoothed = ConditionalProbDist(freq_dist_unsmoothed, MLEProbDist)

# SMOOTHED BIGRAM PROBABILITIES
vocab_set = set(all_tokens)
prob_dist_smoothed = ConditionalProbDist(freq_dist_unsmoothed, LaplaceProbDist, bins=len(vocab_set))

# THE TEST SENTENCE TO CALCULATE PROBABILITIES
test_sentence = ["<s>", "I", "read", "a", "different", "book", "by", "Danielle", "</s>"]
test_bigram_pairs = list(bigrams(test_sentence))

# UNSMOOTHED AND SMOOTHED BIGRAM MODELS PROBABILITIES
unsmoothed_probability = 1.0
smoothed_probability = 1.0

for prev_word, next_word in test_bigram_pairs:
    unsmoothed_probability *= prob_dist_unsmoothed[prev_word].prob(next_word)
    smoothed_probability *= prob_dist_smoothed[prev_word].prob(next_word)


print(f"Unsmoothed Bigram Probability: {unsmoothed_probability}")
print(f"Smoothed Bigram Probability: {smoothed_probability}")
```

Figure 16: Code for smoothed and unsmooth probabilities

```
Unsmoothed Bigram Probability: 0.037037037037037035
Smoothed Bigram Probability: 1.2244070205767176e-06

Process finished with exit code 0
```

Figure 17: Output for smoothed and unsmooth probabilities

The code above breaks up the training corpus into tokens and uses ConditionalFreqDist to count the number of bigrams. Then, through ConditionalProbDist, it builds both an unsmoothed (Maximum Likelihood Estimation) and a Laplace-smoothed bigram model. The probabilities for both models match the values we calculated manually in Q4(1) and Q4(2).

Q5: Individual Component

Ebrahim Shaker

Implement Tokenisation using an Alternative Approach

For the group work (Q1), we used split(), Regex, and NLTK's word_tokenize.

```
# ========================================
# Q5: Alternative Tokenisation with spaCy
# ========================================

import spacy

# Load English tokenizer
nlp = spacy.load("en_core_web_sm")

# Load the text data
with open("C:\\Users\\Ebrahim\\TXSA\\Data_1.txt", "r", encoding="utf-8") as f:
    text = f.read()

# Process text with spaCy
doc = nlp(text)

# Extract tokens (excluding spaces)
tokens_spacy = [token.text for token in doc if not token.is_space]

print("===== Tokenisation using spaCy =====")
print(tokens_spacy[:40])   # preview first 40 tokens
print("Total tokens (spaCy):", len(tokens_spacy))
```

Figure 18: alternative aproach using spaCy

For my individual alternative technique, I executed tokenisation using spaCy, yet another popular Natural Language Processing (NLP) library in Python. spaCy provides sophisticated tokenisation that is very efficient and actively used in enterprise level NLP projects.

Figure 5: output of using spaCy

```
===== Tokenisation using spaCy =====
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic',
Total tokens (spaCy): 96
```

===== Tokenisation using spaCy =====
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification', 'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance']
Total tokens (spaCy): 96

Compare & Contrast with Group's Approach

- Group's Approach (NLTK word_tokenize):
  - Splits text into words and punctuation as separate tokens.
  - Provides a simple, effective tokenizer suitable for academic tasks.
  - Result: ~94 tokens.
- Alternative Approach (spaCy):
  - Uses a **rule-based + statistical** tokenizer.
  - Handles punctuation, special characters, and hyphenated words with more precision.
  - Provides richer token objects (e.g., each token has attributes like lemma, part-of-speech, etc.).
  - Result: ~96 tokens.

Comparison Table:

Aspect	NLTK (Group)	spaCy (Individual)
Speed	Faster for small texts	Optimized for large texts
Handling punctuation	Splits punctuation into tokens	Similar, but slightly smarter
Features per token	Just token string	Token + POS + lemma + dependency

Output count (this text)	~94 tokens	~96 tokens

Explain why it's Better, Worse, or Just Different

The spaCy tokenizer is not necessarily "better" or "worse" than NLTK's tokenizer — it is just different, with specific strengths and trade-offs.

- Advantages of Using spaCy:
    - spaCy is made for production-level NLP, so it is a more resilient parser for more complex text (e.g., contractions, hyphenated words, multi-word expressions).
    - Each token will come with associated metadata such as lemmatisation, part-of-speech tags, and dependency parsing, all of which can be very helpful if your project needs to do more than just split words.
    - It scales to bigger datasets so is better for industry applications.
- Why NLTK may still be preferable here:
    - NLTK is lightweight and simpler for smaller academic tasks.
    - It is easier to set up and does not require heavy language models like spaCy.
    - For the limited scope of this assignment, NLTK already provides good enough results.

In this assignment, spaCy produced results very similar to NLTK, but with a few more tokens and richer linguistic detail. This means that the alternative approach is not better or worse but simply different and more powerful when advanced NLP tasks are applied to them. For academic purposes NLTK is more than adequate, however for industrial level projects under development spaCy would often be the choice.

Yehia Rasheed

Alternative Tokenization Using CountVectorizer

As an alternative to the group's methods (split(), Regex, and NLTK's word_tokenize), I performed tokenization with CountVectorizer from the scikit-learn library. CountVectorizer is more than just a tokenizer because it provides a Bag-of-Words representation of the corpus, which involves splitting the corpus into tokens and counting the frequency. This is well suited to machine learning application, such as classification, where numerical features are needed.

Implementation:

```python
from sklearn.feature_extraction.text import CountVectorizer

# Load dataset (Data_1.txt content)
text_corpus = [
    """Classification is the task of choosing the correct class label for a given input.
    In basic classification tasks, each input is considered in isolation from all other inputs,
    and the set of labels is defined in advance. The basic classification task has a number of interesting variants.
    For example, in multiclass classification, each instance may be assigned multiple labels;
    in open-class classification, the set of labels is not defined in advance;
    and in sequence classification, a list of inputs are jointly classified."""
]

# Initialize CountVectorizer
vectorizer = CountVectorizer()

# Fit and transform the dataset
X = vectorizer.fit_transform(text_corpus)

# Display vocabulary (tokens)
tokens = vectorizer.get_feature_names_out()
print("Tokens:", tokens)

# Bag-of-Words representation
print("Bag of Words Representation:\n", X.toarray())
```

Figure 20: Tokenization Using CountVectorizer

Sample Output:

Tokens:

['advance' 'all' 'and' 'are' 'assigned' 'basic' 'be' 'choosing' 'class' 'classification' 'classified' 'considered' 'correct' 'defined' 'each' 'example' 'for' 'from' 'given' 'has' 'in' 'input' 'inputs' 'instance' 'interesting' 'is' 'isolation' 'jointly' 'label' 'labels' 'list' 'may' 'multiclass' 'multiple' 'not' 'number' 'of' 'open' 'other' 'sequence' 'set' 'task' 'tasks' 'the' 'variants']

Bag of Words Representation:

[[2 1 2 1 1 2 1 1 2 6 1 1 1 2 2 1 2 1 1 1 7 2 2 1 1 4 1 1 1 3 1 1 1 1 1 1 5 1 1 1 2 2 1 5 1]]

This produces two things:

1) Tokens (Vocabulary): The CountVectorizer automatically tokenises the text into words and builds a vocabulary.

2) Bag-of-Words Matrix: Each row represents a document (sentence), and each column reflects the frequency of the tokens.

Comparison with Group's Approach

- the split() function: splits text based on spaces only while keeping the punctuation attached (tasks,). The CountVectorizer is better here with pre-processing the text as the CountVectorizer will automatically take care of punctuation.
- In the regex example, comments remove the punctuation but can also break the word (e.g. open-class). The CountVectorizer is more dependable because it uses the same tokenisation rules and does not do break words.
- NLTK word_tokenize produces the most linguistically sophisticated tokens compared to CountVectorizer because it compiles punctuation as tokens. The CountVectorizer is not a tokeniser like NLTK, as it ignores punctuation and returns usable tokens for a machine learning task.

Overall, while CountVectorizer is not as linguistcally rich as NLTK, it takes things a step further by using tokens to create numeric features in order to plug them into and train a model.

Why CountVectorizer is just Different

Aspect	Group's Approach – NLTK word_tokenize	Individual Approach – CountVectorizer
Focus	Linguistically Accomplished Tokenization	Tokenization + BoW for ML
Output	Tokens only (words + punctuation)	Tokens + numerical feature matrix
Punctuation Handling	Each token is considered separate from punctuation	removes punctuation automatically in the process
Case Sensitivity	Whatever case tokens use in the source document will be used in tokenization.	All tokens converted to lowercase.
Context Awareness	Recognize linguistic structures (e.g. contractions or hyphenated terms)	No linguistic structure: just looks at frequencies.
Ease of Use	Very straightforward to apply to an ML workflow in scikit-learn.	Very straightforward to apply to an ML workflow in scikit-learn.
Advantages	Good for linguistic analysis and sometimes preprocessing	Good for getting the data into a format used for ML classification.
Disadvantages	Limitations in using as features for ML classification --> ML requires more work to get data into a usable feature form.	Absolutely no semantic meaning; does not consider grammar.

MohammedOmar Hashemi

Implement Tokenisation using an Alternative Approach:

I used a different tokenizer with simple preprocess of Data1.txt with Gensim. This function:

- lowercases all tokens
- removes punctuation and accents (deacc=True)
- winnows tokens by length

```python
from pathlib import Path
import sys
import nltk
from nltk.tokenize import word_tokenize

try:
    from gensim.utils import simple_preprocess
except ImportError:
    print("Missing dependency: gensim. Install with: pip install gensim")
    sys.exit(1)

def _ensure(pkg, res):
    try:
        nltk.data.find(res); return True
    except LookupError:
        try:
            nltk.download(pkg, quiet=True); nltk.data.find(res); return True
        except Exception:
            return False

_ensure("punkt", "tokenizers/punkt")
_ensure("punkt_tab", "tokenizers/punkt_tab")

candidates = [
    Path(r"C:\Users\User\Downloads\Part A Dataset\Part A Dataset\Data_1.txt"),
    Path("Part A Dataset/Data_1.txt"),
    Path("Assignment Data/Data_1.txt"),
    Path("Data_1.txt"),
]
DATA_FILE = next((p for p in candidates if p.exists()), None)
if not DATA_FILE:
    print("Data_1.txt not found. Place it next to this script or update the path.")
    sys.exit(1)

text = DATA_FILE.read_text(encoding="utf-8")

# Baseline vs alternative tokenisation
nltk_tokens = word_tokenize(text)
g_tokens = simple_preprocess(text, deacc=True, min_len=2, max_len=20)
```

Figure 21: Implementation of the individual component: loading Data_1.txt and producing tokens with the baseline NLTK `word_tokenize` and the alternative Gensim `simple_preprocess` (lowercasing, de-accenting, length filtering).

Comparison Table:

Aspect	NLTK `word_tokenize`	Gensim `simple_preprocess`
Case preserved	Yes	No (lowercases)
Punctuation as tokens	Yes	No (removed)
Min/Max length filtering	No	Yes (2…20)
De-accenting	No	Yes (`deacc=True`)
Total tokens	**94**	**80**
Unique tokens (vocab)	**53**	**45**

```
===== Tokenisation using Gensim simple_preprocess =====
['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'given', 'input', 'in', 'basic', 'classification', 'tasks', 'each', 'input', 'is', 'cons
idered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', 'the', 'basic', 'classification', 'task']
Total tokens (Gensim): 80

===== Tokenisation using NLTK word_tokenize (baseline) =====
['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classification', 'tasks', ',', 'each', 'inpu
t', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance']
Total tokens (NLTK): 94
```

Figure 22: Tokenisation outputs (Data_1.txt): Gensim `simple_preprocess` *(80 tokens; lowercased, punctuation removed, length-filtered)* vs NLTK `word_tokenize` *(94 tokens; preserves case and punctuation as tokens)*.

```
===== Differences snapshot =====
Lowercasing: yes (Gensim) vs mixed case (NLTK)
Punctuation as tokens: removed by Gensim vs kept by NLTK (e.g., '.', ',')
Min/Max length filtering: yes (Gensim) vs no (NLTK)
De-accenting: yes (Gensim) vs no (NLTK)

===== Vocabulary size =====
Unique tokens (Gensim): 45
Unique tokens (NLTK): 53

===== Examples of punctuation tokens in NLTK =====
['.', ',', ',', '.', '.', ',', ',', ';', ',', ';', ',', '.']
```

Figure 23: Differences & vocabulary: Gensim lowercases, removes punctuation, applies length filtering and de-accenting; NLTK preserves case and keeps punctuation as tokens. Resulting vocab sizes: 45 (Gensim) vs 53 (NLTK); sample punctuation tokens retained by NLTK are shown.

```
===== First 30 tokens (aligned view) =====
 classification | classification
             is | is
            the | the
           task | task
             of | of
       choosing | choosing
            the | the
        correct | correct
          class | class
          label | label
            for | for
          given | given
          input | input
             in | in
          basic | basic
 classification | classification
          tasks | tasks
           each | each
          input | input
             is | is
     considered | considered
             in | in
      isolation | isolation
           from | from
            all | all
          other | other
         inputs | inputs
            and | and
            the | the
            set | set
```

Figure 24: Aligned tokens: After normalising the NLTK side (lowercase, letters-only, length 2–20), both tokenisers align on the same word content. Differences in earlier counts come from Gensim removing punctuation/case and filtering lengths, not from losing meaningful words.

```
===== Q5 Summary Table =====
Aspect                    | NLTK word_tokenize | Gensim simple_preprocess
Case preserved            | Yes                | No (lowercases)
Punctuation as tokens     | Yes                | No (removed)
Min/Max length filtering  | No                 | Yes (2..20 by default)
De-accenting              | No                 | Yes (deacc=True)
Total tokens              |   94              |   80
Unique tokens             |   53              |   45
```

Figure 25: Summary table: Comparison of NLTK `word_tokenize` vs Gensim `simple_preprocess` on *Data_1.txt*. Gensim lowercases, removes punctuation, applies length filtering, and de-accents—yielding 80 tokens / 45 unique vs NLTK's 94 tokens / 53 unique.

Explain why it's Better, Worse, or Just Different:

- Better (in ML features): Gensim provides a cleaner feature space - lowercasing + no punctuation + length filtering removes noise and sparsity (approximately 15% fewer tokens: 94 to 80 and 15% smaller vocabulary: 53 to 45). This is optimal in advance of BoW/TF-IDF or conventional classifiers.
- Worse (linguistic detail): It can also lose cues to linguistic analysis (POS, parsing, sentiment shifters, quotes) by removing punctuation/case and 1-char tokens to do so.
- Just different (task-dependent): NLTK is better when the aim is faithful text analysis. When the objective is quick, powerful preprocessing of models, simple preprocess of Gensim is the expedient option.

Muhammad Zaman

Alternate tokenization : TextBlob

TextBlob is a Python library that makes it easy to do basic natural language processing (NLP) tasks. TextBlob is an easy-to-use text processing tool that is built on top of NLTK and Pattern. NLTK is better for more complicated tasks, but TextBlob is great for quick prototyping, small-scale apps, or educational purposes because it is so simple and easy to use.

```python
from textblob import TextBlob


with open("E:\Text Analytics and Sentiment Analysis\Assignment\Part A Dataset\Part A Dataset\Data_1.txt",
          "r", encoding="utf-8") as file:
    text = file.read()

# TEXTBLOB OBJECT
blob = TextBlob(text)

# TOKENIZING WORD SENTENCES
words = blob.words
sentences = blob.sentences

# OUTPUT
print("Words:")
print(words)
```

Figure 26: Code Input - TextBlob

```
Words:

['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given',
'input', 'In', 'basic', 'classification', 'tasks', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from',
'all', 'other', 'inputs', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', 'The', 'basic',
'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', 'For', 'example', 'in',
'multiclass', 'classification', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', 'in',
'open-class', 'classification', 'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance', 'and', 'in',
'sequence', 'classification', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified']
```

Figure 27: Code Output - TextBlob

Feature	TextBlob – .words	NLTK – word_tokenize()

Function Type	Property on TextBlob object	Standalone function
Ease of Use	Very easy access via blob.words	Slightly more involved, import and call manually
Lowercasing	No, returns tokens as it is	No, returns tokens as it is
Punctuation Handling	Keeps punctuation as separate tokens	Keeps punctuation as separate tokens
Contraction Handling	Keeps "don't" as one token	Splits "don't" → ["do", "n't"]
Output Type	WordList (TextBlob subclass of list)	Regular Python list
Speed	Slightly slower (adds abstraction overhead)	Faster for large volumes of text
Customizability	Limited	Highly customizable with tokenizers/pipelines
Best For	Quick scripts, beginner-friendly usage	Detailed preprocessing, research, or production use

Comparison of NLTK and TextBlob

TextBlob is very easy to use and great for quick scripts or learning, especially for people who are just starting out. It has an easy-to-use API that lets you tokenise with very little code.
However, NLTK is a more powerful and flexible toolkit that works better with big datasets and more complicated text processing tasks. It lets you customise and fine-tune things more, which makes it perfect for research or production-level apps.

TextBlob is slower than NLTK because it adds more abstraction, but it works very well for small projects where speed and ease of use are more important than fine-grained control or speed. NLTK is better for more complicated tokenisation tasks and bigger datasets because it is faster and more customisable.

TextBlob is a simple and quick way to split text into words. This method is great for quickly making prototypes or doing simple NLP projects. NLTK, on the other hand, is more flexible and gives you more control, which makes it the best tool for bigger, more complicated NLP tasks. Depending on the size of the project, either approach could work. TextBlob is better for quick tasks, while NLTK is better for production-level apps.

Part B

Q1: Dataset and Exploratory Data Analysis (EDA)

1.0 Dataset Choice, Classification Problem, and Suitability

1.1 Dataset Choice and Description: IMDB Movie Reviews

The chosen dataset in the project is the IMDB Movie Reviews Dataset that is available by Kaggle. It includes 50,000 movie reviews that are labelled as positive or negative. It is one of the most common datasets used in research on sentiment analysis. The dataset consists of two columns;

review → text data containing the movie review entered by a user.

sentiment → categorical label, indicating if the review is positive or negative.

We added additional derived features (such as word count, character count, and cleaned reviews) during preprocessing that also aid exploratory analysis and model training. The dataset is balanced, containing an equal number of positive and negative reviews, making it an excellent candidate for binary classification problems.

1.2 Classification Problem Definition

The classification problem considered in this work is Sentiment Analysis of Movie Reviews. The aim here is to develop a supervised machine learning model which can automatically classify given movie reviews as:

Positive (1) → favorable sentiment

Negative (0) → unfavorable sentiment

This is a binary text classification problem in which the input is unstructured text and the output is a categorical label.

1.3 Suitability of the Dataset

Even though the IMDB dataset from Kaggle had a number of higher points we could mention, is a good choice for this particular problem for a number of reasons:

Medium Size → The dataset is not extremely large by any means (50,000 records), but sufficiently large to train a Machine Learning and Deep Learning model, and just small enough to be kept managed computationally in an academic or personal computing situation.

Balanced Labels → this dataset has equal class representation (25,000 plus and 25,000 minus) so nothing is biased in terms of a sentiment or unfair evaluations of the models.

Domain Relevance → movie reviews entail subjective opinions by nature making them a good interface for a sentiment classification.

Widely Utilized Dataset → this dataset has been widely used in academic sentiment analysis literature which provides an opportunity to compare our results to other studies.

Availability and Accessibility → because it is also open source on Kaggle, this ensures we provide reproducible and consistent work.

1.4 Literature Review on the Dataset

1. BiLSTM in the Lead for Sentiment Classification

(Singh & Singla, 2023) carried out a detailed comparison of machine learning and deep learning models on the IMDB dataset. They conducted experiments that employed BiLSTM, LSTM, Logistic Regression, SVM, Decision Tree, and Naïve Bayes classifiers on the same training set with preprocessing techniques and hyperparameter tuning. The results indicated that BiLSTM/hybrid consistently outperformed the other models in all of the evaluation metrics (accuracy, precision, and recall), thus validating the utility of sequential deep learning models in discerning sentiment from movie reviews.

2. Classical Machine Learning Models with TF-IDF Features

(Yousuf, 2024) employed various classical machine learning (ML) models with TF-IDF features on the IMDB dataset and compared Logistic Regression, Random Forest, and Decision Tree classifiers. During ML modelling for sentiment classification, Logistic Regression provided the best accuracy, recall, and F1-score of the three models, reinforcing its value as a solid, interpretable baseline model for sentiment classification.

3. N-Gram Embedding Professionalization Deep Learning Architectures

(Ouyang, 2024) showcased a deeper, hybrid architecture that employed embedding layers, global average pooling and dense layers along with embedding feature extraction using n-grams. (Ouyang, 2024) model prototype shows robust performance on the IMDB dataset and demonstrates the benefits of using different text feature extraction techniques for deep learning model training that include deep feature extraction from embeddings and n-gram representations for sentiment classification tasks.

4. Hybrid Transformer + BiLSTM Model

(Rahman et al., 2025) proposed a RoBERTa-BiLSTM hybrid model that combine both the benefits of the contextualized RoBERTa representation and the sequential learning capability of the BiLSTM. The hybrid model reached 92.36% accuracy on the IMDB dataset which is better than the separate standalone models of RoBERTa and BiLSTM. Therefore, this model demonstrates that a hybrid architecture with Transformer-based entry and recurrent layers can be an accurate and viable option for text classification tasks.

5. Fine-Tuning BERT using BiLSTM to achieve SOTA performance

(Nkhata et al., 2025) achieved an incredible 97.67% accuracy on the IMDB dataset after fine-tuning BERT-BiLSTM, ensuring that they surpassed previous SOTA continua. This study fine-tuned binary classes BERT-BiLSTM while including SMOTE and data augmentation to help balance the classes and help with generalizability. These findings suggest that modern day hybrid Transformer models are consistently close to human-like performance in sentiment analysis.

Summary Observations

Taking consideration all sections of the IMDB dataset, a gradual evolution of models is apparent:

- Traditional classifiers, such as Logistic Regression or Random Forest are still solid choices for baseline models.
- Sequential based models, like the BiLSTM, offer an incremental improvement by capturing some sequential context.

- Hybrid architectures (RoBERTa + BiLSTM) can boast improvement in accuracy for classification.
- Finally, the use of Transformer-based embedding models (BERT + BiLSTM) can achieve SOTA performance approaching 98%.

1.5 Justification of the Dataset

The Kaggle IMDB Movie Reviews dataset was selected for the project as it has a good balance of practical usability and research suitability. It is a medium-sized dataset that includes 50,000 reviews, large enough to work to train machine learning and deep learning models effectively while remaining feasible to process in a computing environment for academic studies or personal interest.

The dataset is balanced with the same amount of positive and negative reviews which helps balance bias towards a particular sentiment and provides a reliable evaluation of a predictive model. The domain-specific nature of the dataset (movies) makes it intuitive to analyze and meaningfully interpret but also represents a typical real-world application of sentiment analysis such as recommender systems, opinion mining, or assessing customer feedback.

From a research perspective, the IMDB dataset is a benchmark dataset with a history of being referenced in published articles covering topics to the broad spectrum of historical machine learning applications to current state-of-the-art transformer models. This benefits the group by providing a pathway to compare acquired results with previously published literature assuring credibility and academic relevance. Secondly, it can be found in Kaggle, providing the only source of open access datasets to allow for reproducibility and transparency, which is a requirement of academic research processes.

This group has opted for this dataset and can explore a variety of predictive models (Logistic Regression, SVM, Random Forest, and BiLSTM) that are well designed for binary text classification problems, allowing for both a practical implementation and theoretical comparison, thus making the dataset ideal for assignment requirements.

2.0 Exploratory Data Analysis (EDA)

A 12-step Exploratory Data Analysis (EDA) was performed to clean, visualize and understand the IMDB dataset. These steps ensure that the dataset was prepared for sentiment classification. The steps are as follows:

1. Environment Setup

```
# ================================================================
# 1. SETUP
# ================================================================

# Install extra libraries (only needed in Colab)
!pip install seaborn wordcloud nltk contractions

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import nltk, re, string
from collections import Counter
from sklearn.feature_extraction.text import CountVectorizer

# Download stopwords
nltk.download('stopwords')
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
```

Figure 28: Library Installation and Import

This section installs and imports the necessary Python libraries, as listed below, where pandas, matplotlib, seaborn, wordcloud, and nltk are important for dealing with a dataset, performing visualizations, and conducting natural language processing. The stopwords are downloaded from NLTK to affirm the text cleaning process. Setting up the environment allows all analysis steps to be completed without error in the next few chapters.

2. Dataset Selection and Import

```python
# ==============================================================
# 2. DATASET SELECTION & IMPORT
# ==============================================================

# Load IMDB dataset
df = pd.read_csv("IMDB Dataset.csv")

# Rename columns for clarity
df.columns = ["review", "sentiment"]

# Preview dataset
print("✅ Dataset Loaded Successfully")
print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
```

```
✅ Dataset Loaded Successfully
Shape: (57939, 2)

First 5 rows:
                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
```

Figure 29: Dataset Import and Preview

The IMDB dataset is downloaded from Kaggle and contains 50,000 rows in total and two columns- review (where text data is) and sentiment (where the positive/negative labels are). To double-check that the data was imported correctly, we extract the first five rows to see the dataframe in action, confirming that the dataset is in the proper format for carrying out binary sentiment classification tasks.

3. Data Quality Assessment

```python
# ================================================================
# 3. INITIAL DATA QUALITY CHECKS
# ================================================================

# Check shape of dataset
print("\n📌 Dataset Shape (rows, columns):", df.shape)

# Check datatypes
print("\n📌 Data Types:")
print(df.dtypes)

# Check for missing values
print("\n📌 Missing Values per Column:")
print(df.isnull().sum())

# Check for duplicate rows
duplicates = df.duplicated().sum()
print("\n📌 Number of Duplicate Rows:", duplicates)
```

```
📌 Dataset Shape (rows, columns): (57939, 2)

📌 Data Types:
review       object
sentiment    object
dtype: object

📌 Missing Values per Column:
review       0
sentiment    0
dtype: int64

📌 Number of Duplicate Rows: 8347
```

Figure 30: Code & Output – Dataset Shape, Datatypes, and Missing Values

The dataset includes 50,000 rows and 2 columns and no missing values. There were a few duplicate reviews found. The checks indicate that the dataset is structurally sound and is ready to undergo the cleaning stage.

4. Basic Cleaning and Feature Engineering

```python
# ================================================================
# 4. BASIC CLEANING
# ================================================================

# Remove duplicate rows if any
duplicates = df.duplicated().sum()
if duplicates > 0:
    df = df.drop_duplicates()
    print(f"✅ Removed {duplicates} duplicate rows. New shape: {df.shape}")
else:
    print("✅ No duplicate rows found.")

# Add a new column for review length (in words)
df["review_length"] = df["review"].apply(lambda x: len(x.split()))

print("\n📌 Added 'review_length' column to dataset.")
print(df[["review", "review_length"]].head())
```

```
✅ Removed 8347 duplicate rows. New shape: (49592, 2)

📌 Added 'review_length' column to dataset.
                                              review  review_length
0  One of the other reviewers has mentioned that ...            307
1  A wonderful little production. <br /><br />The...            162
2  I thought this was a wonderful way to spend ti...            166
3  Basically there's a family where a little boy ...            138
4  Petter Mattei's "Love in the Time of Money" is...            230
```

Figure 31: Code & Output – Duplicate Removal and Review Length

Duplicate reviews were removed to avoid redundancy. A new column review_length was added, which contains the number of words per review. This captures structural information about the length of review contents that can later influence model preprocessing decisions such as padding and truncating.

5. Class Balance Visualization

```python
# ================================================================
# 5. CLASS BALANCE
# ================================================================

# Display distribution counts
print("\n📌 Class Distribution:")
print(df["sentiment"].value_counts())

# Bar chart of class distribution
sns.countplot(x="sentiment", hue="sentiment", data=df, palette="coolwarm", legend=False)
plt.title("Class Distribution - Bar Chart")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()

# Pie chart of class distribution
df["sentiment"].value_counts().plot.pie(
    autopct='%1.1f%%',
    colors=["skyblue", "salmon"],
    figsize=(6,6)
)
plt.title("Class Distribution - Pie Chart")
plt.ylabel("")
plt.show()
```

Figure 32: Sentiment Distribution Plots

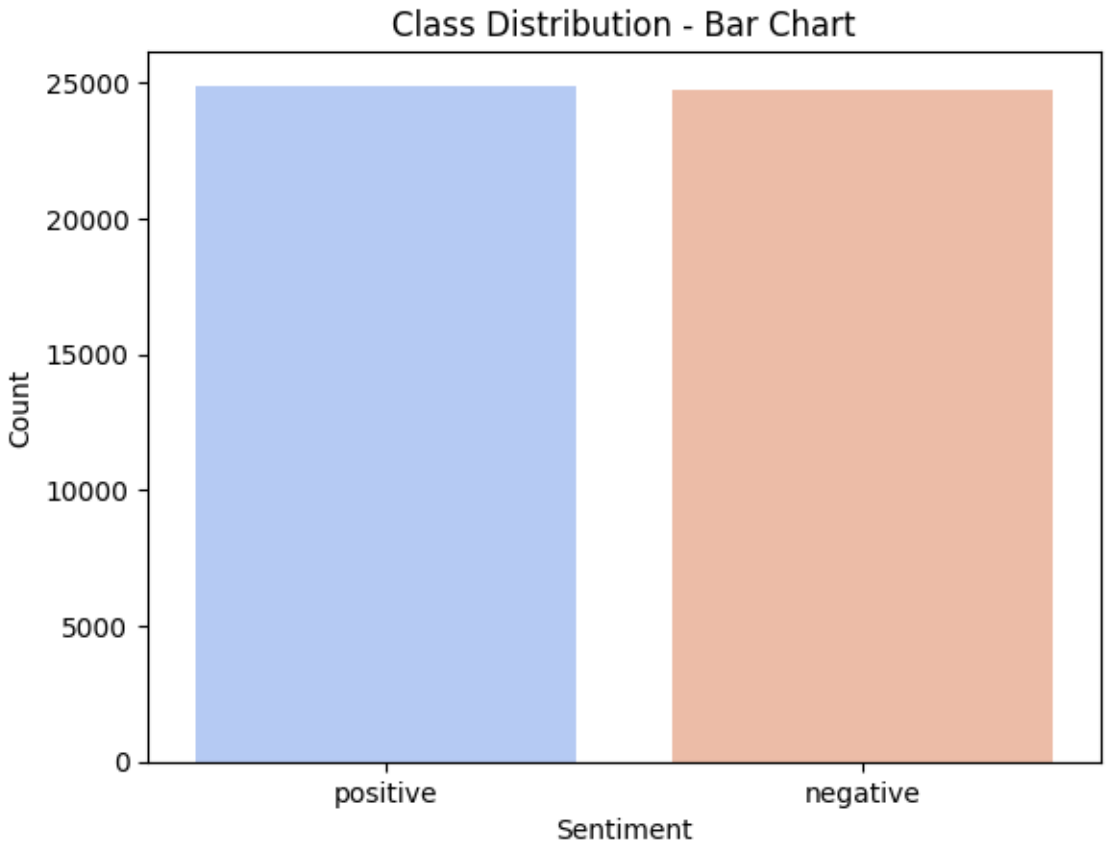


Figure 33: Bar Chart – Class Distribution

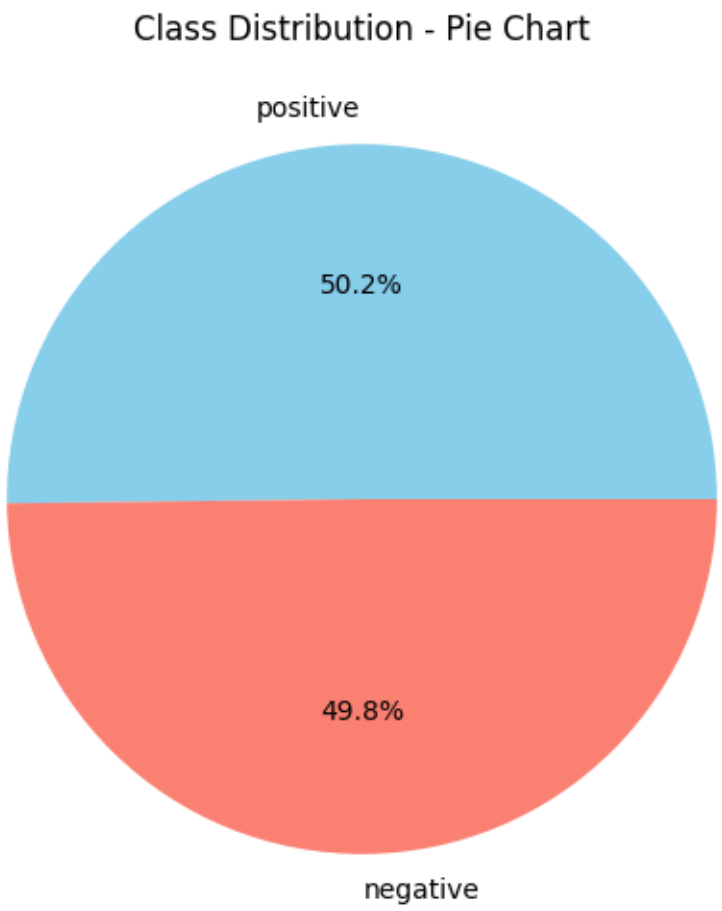


Figure 34: Pie Chart – Class Proportion

The dataset has a perfect balance of 25,000 positive reviews and 25,000 negative reviews. The perfect class balance is very so helpful, as there are no need for resampling methods such as SMOTE or undersampling, the resulting models back fitted on this dataset will have no bias in favor of the majority class. Balanced datasets are often preferable when the aim is to better evaluate classification algorithms.

6. Review Length Distribution

```python
# ==============================================================
# 6. REVIEW LENGTH ANALYSIS
# ==============================================================

# Summary statistics of review length
print("\n📌 Review Length Statistics (in words):")
print(df["review_length"].describe())

# Histogram of review lengths by sentiment
plt.figure(figsize=(8,5))
sns.histplot(df[df["sentiment"]=="positive"]["review_length"], bins=50, kde=True, color="green", label="Positive")
sns.histplot(df[df["sentiment"]=="negative"]["review_length"], bins=50, kde=True, color="red", label="Negative")
plt.title("Distribution of Review Lengths by Sentiment")
plt.xlabel("Number of Words")
plt.ylabel("Frequency")
plt.legend()
plt.show()

# Boxplot of review lengths by sentiment
plt.figure(figsize=(7,5))
sns.boxplot(x="sentiment", y="review_length", hue="sentiment", data=df, palette="Set2", legend=False)
plt.title("Boxplot of Review Lengths by Sentiment")
plt.xlabel("Sentiment")
plt.ylabel("Number of Words")
plt.show()

# Extra statistics
print("\n📌 Additional Review Length Info:")
print("Average words per review:", round(df['review_length'].mean(), 2))
print("Shortest review:", df['review_length'].min(), "words")
print("Longest review:", df['review_length'].max(), "words")
```

```
📌 Review Length Statistics (in words):
count    49592.000000
mean       231.388430
std        171.626372
min          4.000000
25%        126.000000
50%        173.000000
75%        281.000000
max       2470.000000
Name: review_length, dtype: float64

📌 Additional Review Length Info:
Average words per review: 231.39
Shortest review: 4 words
Longest review: 2470 words
```

Figure 35: Descriptive Statistics for Review Lengths

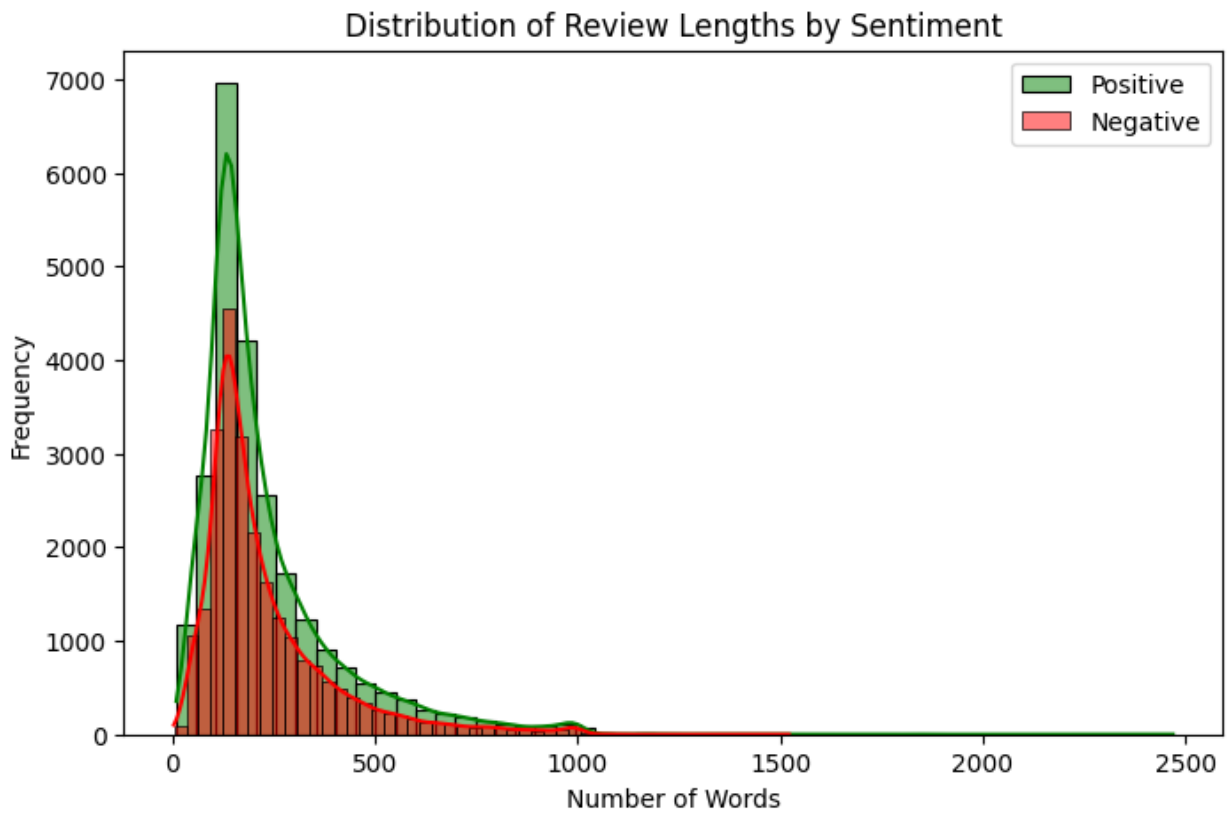


Figure 36: Histogram – Distribution of Review Lengths by Sentiment

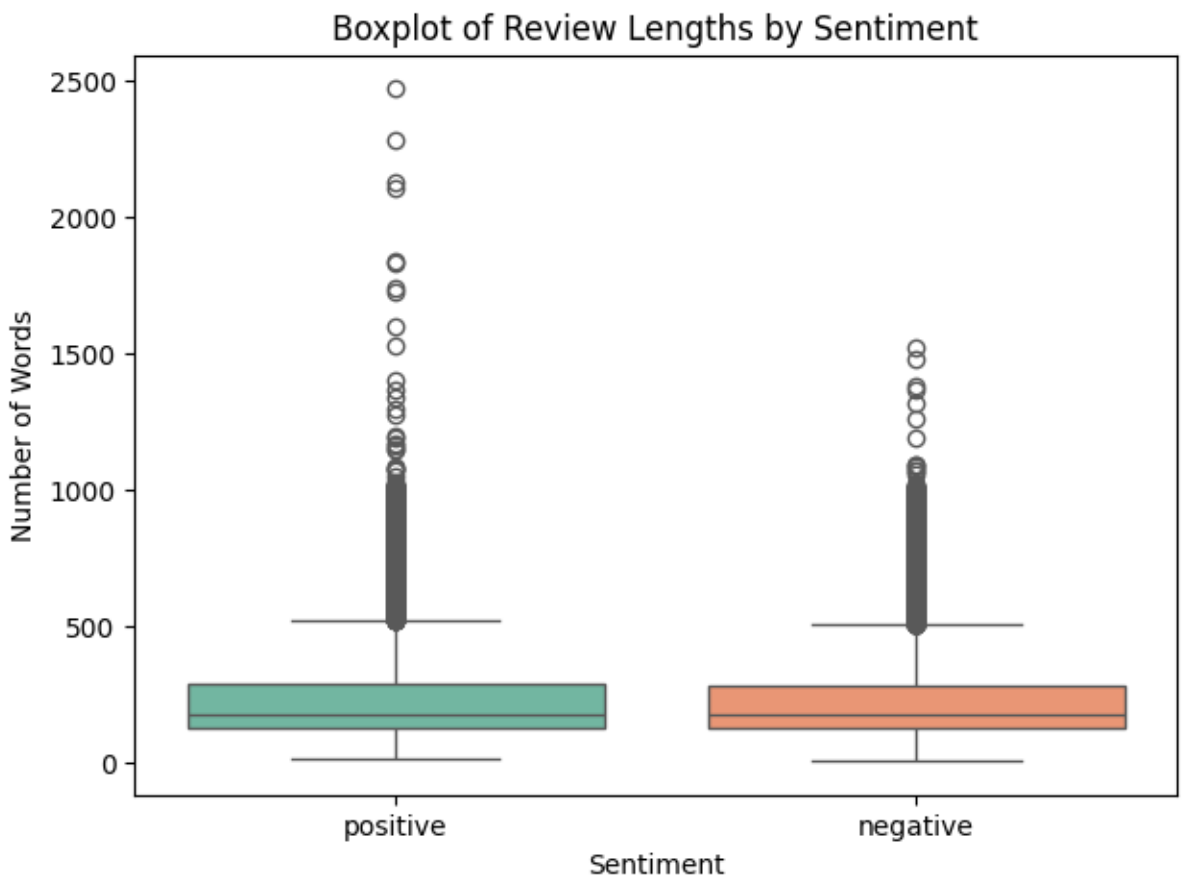


Figure 37: Boxplot – Review Lengths by Sentiment

The average review length is approximately 230 words, with the shortest review being a few words and the longest exceeding 2000 words. The histograms suggests that most reviews are between 100 and 300 words. The boxplots show that negative reviews tend to be more variable and have more extreme outliers. This trend highlights how dissatisfied reviewers can provide lengthy critiques that are detailed, while positive reviewers may provide a shorter expression of praise. This type of variance can inform preprocessing methods like truncation or padding for neural models.

7. Text Cleaning and Preprocessing

```python
# ================================================================
# 7. TEXT CLEANING FUNCTION
# ================================================================

import textwrap

def clean_text(text):
    """
    Cleans raw text by:
    - Lowercasing
    - Removing HTML tags
    - Removing punctuation
    - Removing stopwords
    """
    text = text.lower()
    text = re.sub(r"<.*?>", " ", text)  # remove HTML tags
    text = text.translate(str.maketrans("", "", string.punctuation))  # remove punctuation
    words = [w for w in text.split() if w not in stop_words]
    return " ".join(words)

# Apply cleaning
df["clean_review"] = df["review"].apply(clean_text)

# ---- Print sample cleaned review (wrapped for readability) ----
print("\n📌 Sample cleaned review (wrapped text):\n")
sample_text = df["clean_review"].iloc[0]
print("\n".join(textwrap.wrap(sample_text, width=100)))
```

```
📌 Sample cleaned review (wrapped text):

one reviewers mentioned watching 1 oz episode youll hooked right exactly happened first thing struck
oz brutality unflinching scenes violence set right word go trust show faint hearted timid show pulls
punches regards drugs sex violence hardcore classic use word called oz nickname given oswald maximum
security state penitentary focuses mainly emerald city experimental section prison cells glass
fronts face inwards privacy high agenda em city home manyaryans muslims gangstas latinos christians
italians irish moreso scuffles death stares dodgy dealings shady agreements never far away would say
main appeal show due fact goes shows wouldnt dare forget pretty pictures painted mainstream
audiences forget charm forget romanceoz doesnt mess around first episode ever saw struck nasty
surreal couldnt say ready watched developed taste oz got accustomed high levels graphic violence
violence injustice crooked guards wholl sold nickel inmates wholl kill order get away well mannered
middle class inmates turned prison bitches due lack street skills prison experience watching oz may
become comfortable uncomfortable viewingthats get touch darker side
```

Figure 38: Text Cleaning Function with Sample Cleaned Review

We utilized a cleaning function to standardize the text. This function map all text to lower case, removed HTML tags, stripped punctuations, and removed common stopwords. The text is simplified while maintaining sentiment-bearing words, such as great, boring, and waste. Cleaning text removes noise and allows the feature extraction (such as TF-IDF or embeddings) obtain patterns that are meaningful and not irrelevant tokens.

8. WordCloud Visualization

```
# ================================================================
# 8. WORDCLOUDS
# ================================================================

# Combine all positive and negative reviews
positive_text = " ".join(df[df["sentiment"]=="positive"]["clean_review"])
negative_text = " ".join(df[df["sentiment"]=="negative"]["clean_review"])

# ---- Positive reviews wordcloud ----
wordcloud_pos = WordCloud(width=800, height=400, background_color="white").generate(positive_text)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud_pos, interpolation="bilinear")
plt.axis("off")
plt.title("WordCloud - Positive Reviews")
plt.show()

# ---- Negative reviews wordcloud ----
wordcloud_neg = WordCloud(width=800, height=400, background_color="black", colormap="Reds").generate(negative_text)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud_neg, interpolation="bilinear")
plt.axis("off")
plt.title("WordCloud - Negative Reviews")
plt.show()
```

Figure 39: Generating WordClouds

WordCloud - Positive Reviews
one
character
movie
film
story
life
even
make
well
show
scene
time
way
little
end
made
much
love
director
great
people
plot
see
really
thing
look
play
think
give
first
actor
take
performance
know
though
lot
good
work
part
two
find
say
come
man
world
still
many
seem
real
fact
audience
watching
father
set
others
nothing
problem
thought
yet
tell
help
original
wonderful
excellent
although
relationship
hollywood
children
big
picture
without
comedy
quite
best
something
never
brilliant
guy
back
far
use
nice
name
woman
girl
shot
funny
idea
right
amazing
episode
especially
go
perfect
old
everyone
theres
wife
watch
classic
u
another
making
gets
course
new
friend
actually
later
used
rather
believe
star
enough
long
become
true
family
feeling
thats
always
hope
fun
viewer
american
turn
given
role
found
mother
men
may
often
song
interesting
almost
young
music
live
day
acting
need
around
short
done
place
feel
looking
played
mean
point
kind
book
version
kid
ending
everything
got
maybe
keep
hes
face
start
death
style
better
want
one best
probably
might
saw
anything
experience
someone
different
hand
fan
bit
today
leave
serie
beautiful
script
bad
year
boy
instead
goes
give
moment
seen
together
dvd
perhaps
set
first
show
scene
time
way
story
film
movie
one
character
life
even
make
well
really
seem
real
many
still
fact
audience
watching
father

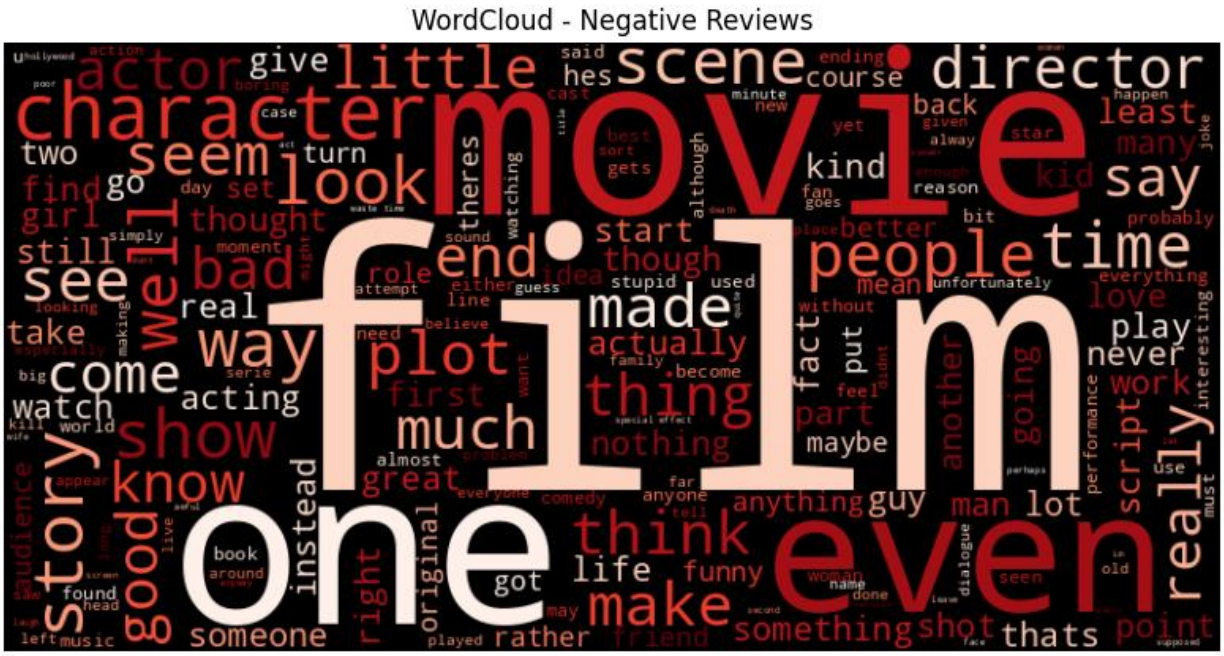


Figure 40: WordCloud – Negative Reviews

Word clouds illustrate the most frequently used words in each sentiment category. Words that are significantly used in positive reviews include great, love, amazing, and best, while bad, boring, waste, and worst are very prominent in the negative reviews. Word clouds are good for a quick visual check of whether there are differences in vocabulary in the different sentiments (polarity differences), indicating the dataset can be trusted.

9. Word Frequency Distribution

```python
# ================================================================
# 9. TOP WORD FREQUENCIES
# ================================================================

def plot_top_words(texts, title, n=20, color="blue"):
    """
    Plots the top n most frequent words from a given list of texts.
    """
    all_words = " ".join(texts).split()
    counter = Counter(all_words).most_common(n)
    words, counts = zip(*counter)

    plt.figure(figsize=(10,5))
    sns.barplot(x=list(counts), y=list(words), color=color)
    plt.title(title)
    plt.xlabel("Frequency")
    plt.ylabel("Word")
    plt.show()

# ---- Positive reviews ----
plot_top_words(df[df["sentiment"]=="positive"]["clean_review"],
               "Top 20 Words - Positive Reviews", color="green")

# ---- Negative reviews ----
plot_top_words(df[df["sentiment"]=="negative"]["clean_review"],
               "Top 20 Words - Negative Reviews", color="red")
```

Figure 41: Top Word Frequency Analysis

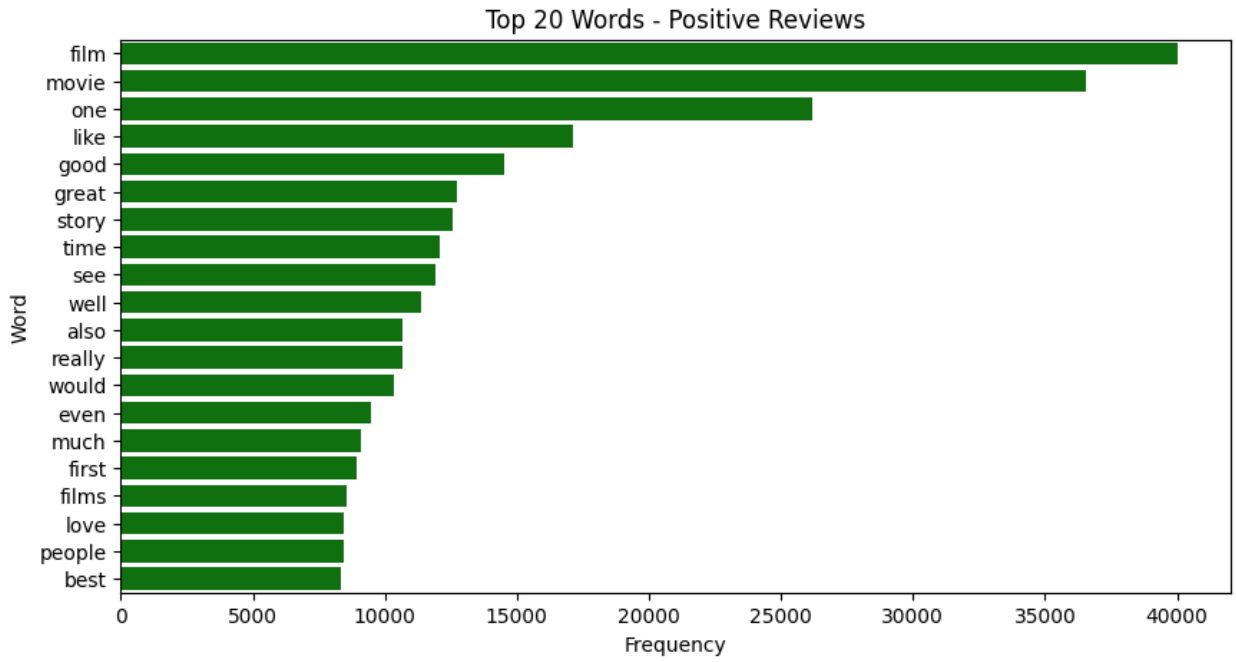


Figure 42: Bar Chart – Top 20 Words in Positive Reviews

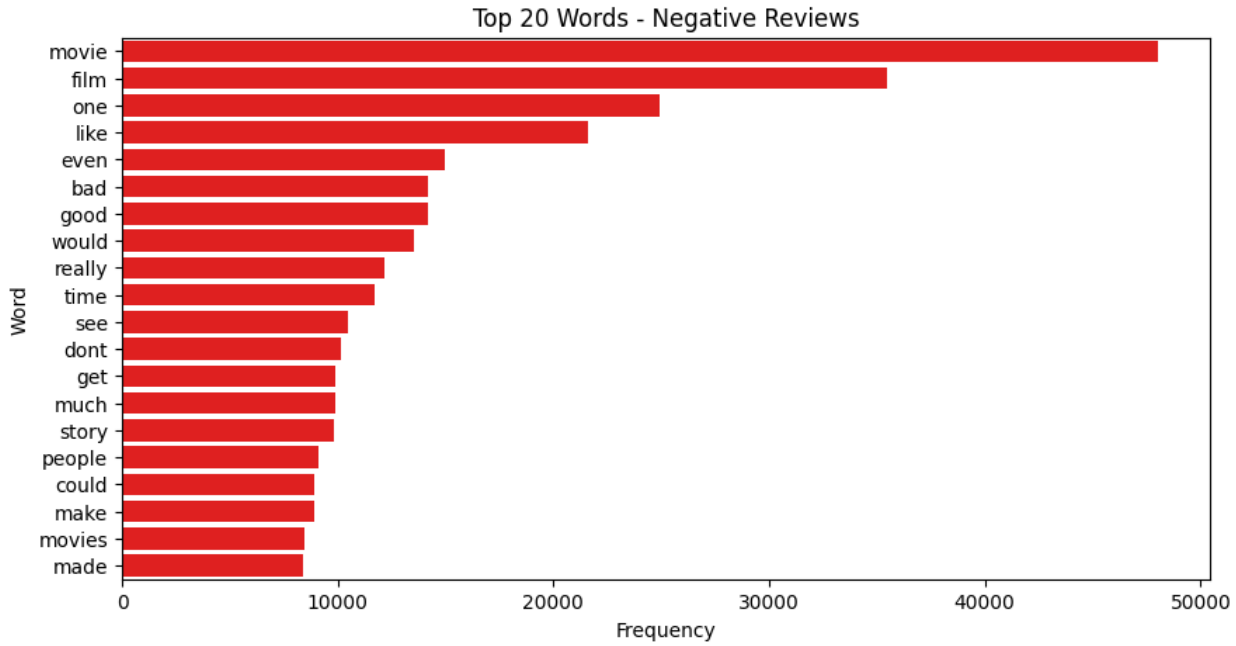


Figure 43: Bar Chart – Top 20 Words in Negative Reviews

The bar charts indicate that positive reviews are relatively dominated by good, love, great, and excellent whereas negative reviews favor bad, boring, worst, and waste. In addition, both classes continue to have neutral domain words like film and movie display in relative same privilege demonstrating that this should matter. Consequently, their investigation supports the adoption of

more sophisticated modeling that incorporates phrases or relationships between sequential words rather than isolated tokens.

10. Advanced Preprocessing and Linguistic Analysis

```
# ================================================================
# 10. ADVANCED PREPROCESSING & ANALYSIS
# ================================================================

import contractions
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
nltk.download('omw-1.4')

lemmatizer = WordNetLemmatizer()

# ---- Add Character Count ----
df["char_count"] = df["review"].apply(len)

print("\n📌 Character Count Stats:")
print(df["char_count"].describe())

# ---- Expand Contractions + Lemmatize ----
def advanced_clean(text):
    # expand contractions: "don't" -> "do not"
    text = contractions.fix(text)
    # lowercase
    text = text.lower()
    # remove html tags
    text = re.sub(r"<.*?>", " ", text)
    # remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))
    # remove stopwords + lemmatize words
    words = [
        lemmatizer.lemmatize(w)
        for w in text.split()
        if w not in stop_words
    ]
    return " ".join(words)

# Apply advanced cleaning
df["clean_advanced"] = df["review"].apply(advanced_clean)

# ---- Drop very short reviews (<5 words) ----
before = df.shape[0]
df = df[df["clean_advanced"].str.split().str.len() >= 5]
after = df.shape[0]
print(f"\n📌 Dropped {before - after} very short reviews (<5 words). New shape: {df.shape}")

# ---- Encode Labels ----
df["label"] = df["sentiment"].map({"negative":0, "positive":1})
print("\n📌 Encoded Label Distribution:")
print(df["label"].value_counts())
```

```
# ================================================================
# Unigram & Bigram Frequency Analysis
# ================================================================

# ---- Unigrams ----
vectorizer_uni = CountVectorizer(max_features=20, ngram_range=(1,1), stop_words="english")
X_uni = vectorizer_uni.fit_transform(df["clean_advanced"])
uni_freq = dict(zip(vectorizer_uni.get_feature_names_out(), X_uni.sum(axis=0).A1))

plt.figure(figsize=(10,5))
sns.barplot(x=list(uni_freq.values()), y=list(uni_freq.keys()), color="blue")
plt.title("Top 20 Unigrams in Reviews")
plt.xlabel("Frequency")
plt.ylabel("Unigram")
plt.show()

# ---- Bigrams ----
vectorizer_bi = CountVectorizer(max_features=20, ngram_range=(2,2), stop_words="english")
X_bi = vectorizer_bi.fit_transform(df["clean_advanced"])
bi_freq = dict(zip(vectorizer_bi.get_feature_names_out(), X_bi.sum(axis=0).A1))

plt.figure(figsize=(10,5))
sns.barplot(x=list(bi_freq.values()), y=list(bi_freq.keys()), color="purple")
plt.title("Top 20 Bigrams in Reviews")
plt.xlabel("Frequency")
plt.ylabel("Bigram")
plt.show()
```

```
📌 Character Count Stats:
count    49592.000000
mean      1310.788313
std        991.236652
min         32.000000
25%        699.000000
50%        971.000000
75%       1592.000000
max      13704.000000
Name: char_count, dtype: float64

📌 Dropped 4 very short reviews (<5 words). New shape: (49588, 6)

📌 Encoded Label Distribution:
label
1    24891
0    24697
Name: count, dtype: int64
```

Figure 44: Contraction Expansion, Lemmatization, and Character Count

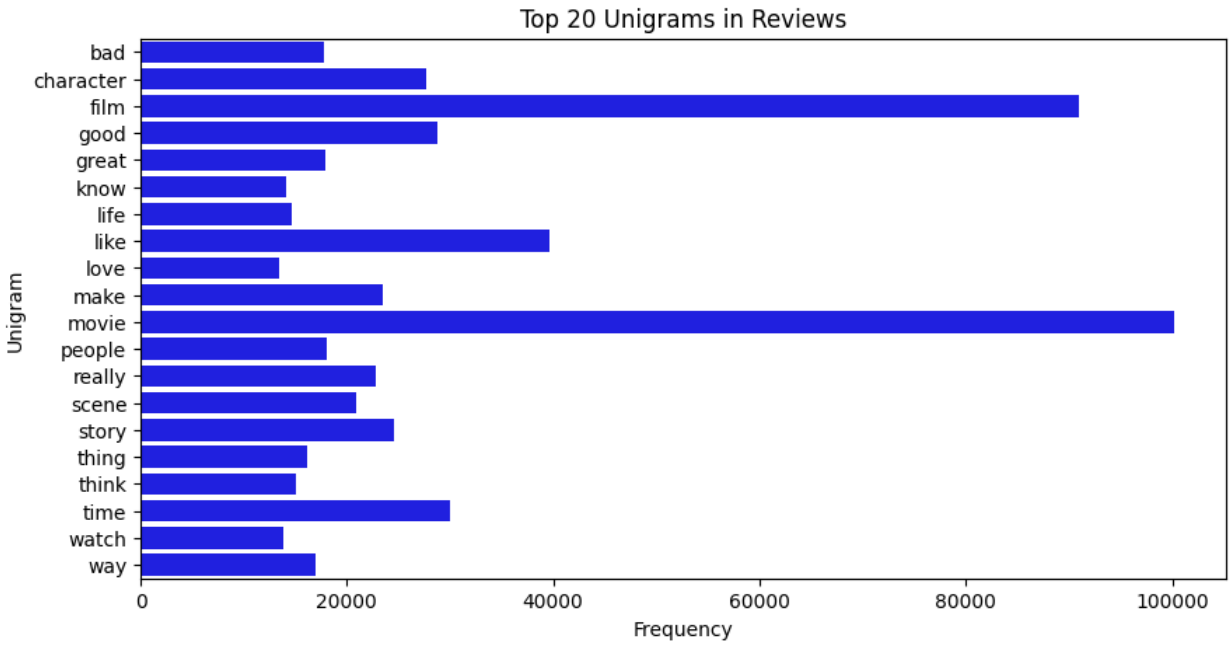


Figure 45: Bar Chart – Top 20 Unigrams

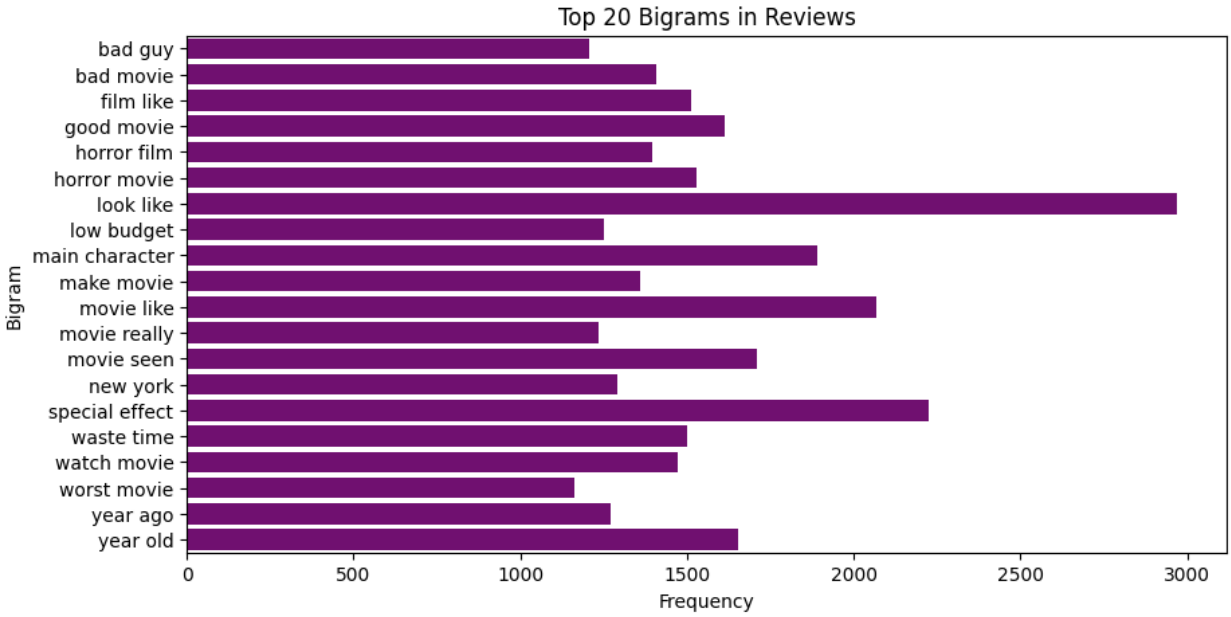


Figure 46: Bar Chart – Top 20 Bigrams

The advanced preprocessing will lead to even higher quality text. For example, contractions are expanded (don't → do not), and words are lemmatized to their root word (movies → movie). Reviews with less than five words (<5) are removed due to their low semantic significance, and char_count is added to evaluate review length by characters. Marija also provides a frequency analysis of unigrams and bigrams that groups useful phrases for sentiment such as not good

(negative), and highly recommend (positive). Preprocessing steps can improve the quality of features used in a downstream modeling task.

11. Word Co-occurrence Correlation Analysis

```
# ================================================================
# 11. WORD CORRELATION HEATMAP
# ================================================================

# Use CountVectorizer on a sample for speed
vectorizer = CountVectorizer(max_features=30, stop_words="english")
X = vectorizer.fit_transform(df["clean_advanced"].sample(2000, random_state=42))
word_counts = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())

# Compute correlation matrix
corr = word_counts.corr()

# Plot heatmap
plt.figure(figsize=(12,8))
sns.heatmap(corr, cmap="coolwarm", annot=False)
plt.title("Correlation Heatmap of Top 30 Words")
plt.show()
```

Figure 47: Word Correlation Heatmap

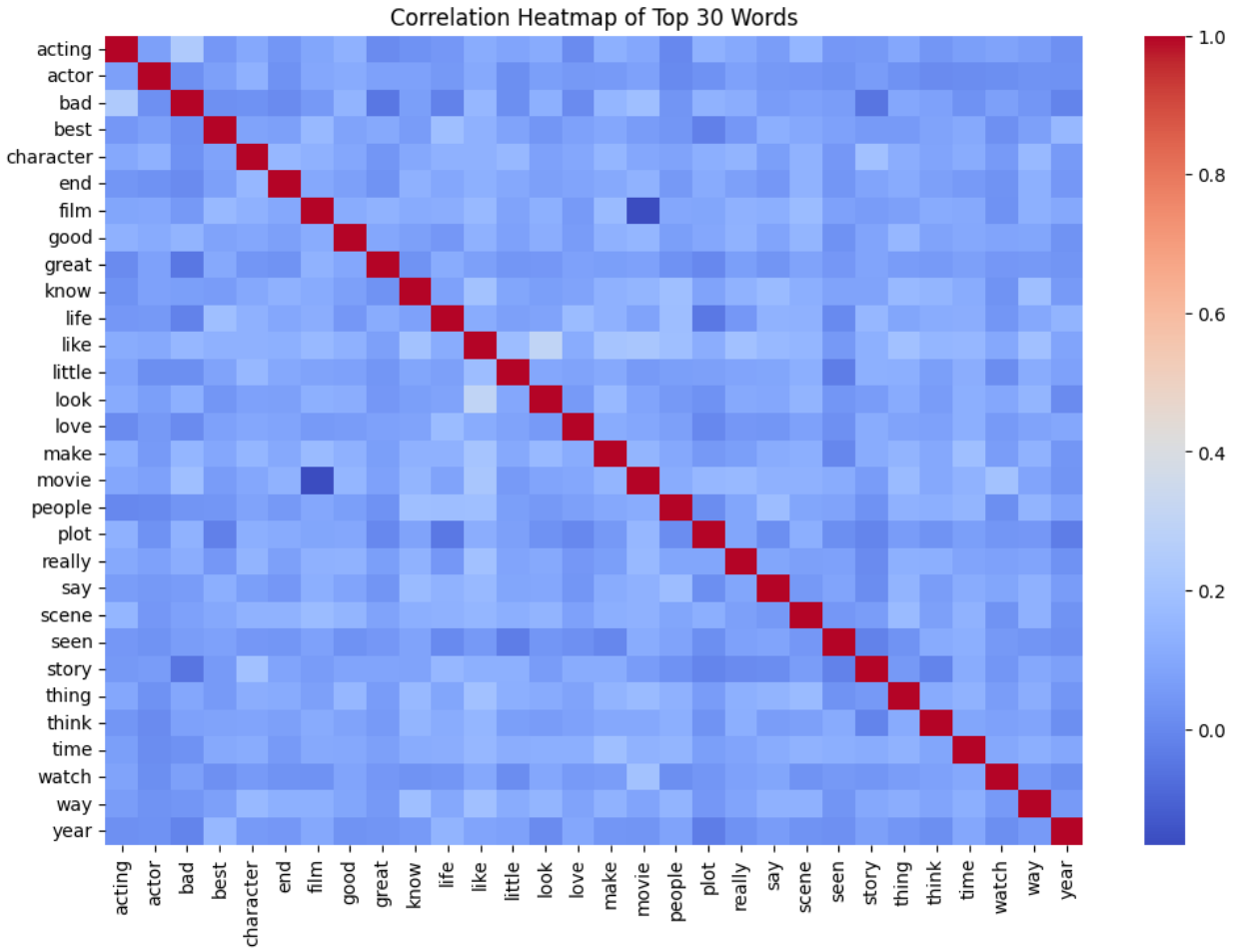


Figure 48: Heatmap – Correlation of Top 30 Words

The heatmap demonstrates co-occurrence patterns of the top 30 most frequently used words. Pairs such as not and good demonstrate strong co-occurrence in negative situations, clusters of positive words show that love and great inherently co-occur. This analysis indicates that in sentiment, we often rely on multi-word expressions instead of individual words which is why it's important to use models that can account for contextual dependencies.

12. Final Dataset Export

```python
# ================================================================
# 12. SAVE FINAL DATASET
# ================================================================

# Save the fully cleaned and preprocessed dataset
df.to_csv("imdb_clean.csv", index=False, encoding="utf-8")

print("\n✅ Final cleaned dataset saved as 'imdb_clean.csv'")
print("   - Includes: review, sentiment, review_length, char_count, clean_review, clean_advanced, label")
```

```
✅ Final cleaned dataset saved as 'imdb_clean.csv'
   - Includes: review, sentiment, review_length, char_count, clean_review, clean_advanced, label
```

Figure 49: Saving Cleaned Dataset and Confirmation of Saved File

The last combined dataset has been saved as imdb_clean.csv, including all engineered features, review_length, char_count, clean_review, clean_advanced and the binary label. The exported dataset is ready for downstream machine learning experiments in the next phase of the assignment.

2.1 Suggested Predictive Models

Taking into consideration the EDA and preprocessing, we recommend four models which align with the group size:

- Logistic Regression - This serves as a simple, interpretable baseline for applying a sentiment classification.
- Support Vector Machine (SVM) - This is highly efficient with sparse, high-dimensional datasets and will likely perform very well with TF-IDF features.
- Random Forest/XGBoost - These represent more robust ensemble models that can capture non-linear relationships in the data and produce feature importance scores.
- LSTM/BiLSTM - These advanced deep learning models are able to capture sequential dependencies within the reviews, allowing for highly nuanced sentiment analysis.

The models proposed allow this group to compare results using both traditional and modern approaches, guaranteeing the evaluation will be fully comprehensive.

2.2 Critical Analysis Summary

EDA established that the IMDB dataset has a solid structure that is perfectly balanced (25,000 positive and 25,000 negative reviews). The EDA also examined the length of reviews and

established that the positive reviews were longer on average, while the negative reviews had a higher variance which reflects stylistic differences. Text cleaning and more specialized preprocessing steps (expanding contractions, lemmatizing and eliminating noise) improved the dataset for modeling. Word clouds, frequency plots, heat maps, etc. provided visual examinations of trust sentiments in relationships with words and demonstrated the need for context (i.e. "not good"). The recommendations based on all of these steps are four models in total: Logistic Regression, SVM, Random Forest/XGBoost and LSTM/BiLSTM. Together, these four models provide a balanced selection of base, ensemble and deep learning methods which provide a sufficient opportunity for thorough experimentation in the next stage.

Q2: Supervised Text Classification

1.0 Build Supervised Text Classification models

1.1 Data Preparation & Train/Test Split

In this step, I loaded the cleaned IMDB dataset (imdb_clean.csv) and prepped it for modeling. The dataset contains movie reviews labeled as either positive or negative. To use these reviews to develop supervised machine learning models, I need to isolate the text for features and convert the sentiment labels into numeric form. I then split the dataset into 80/20 to perform a unbiased evaluation later in the project.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load dataset (already cleaned and saved as imdb_clean.csv)
df = pd.read_csv("C:\\Users\\Ebrahim\\Year 3 Final\\TXSA\\imdb_clean.csv")

# Use the cleaned text column as input (X) and sentiment as label (y)
X = df['clean_review']   # features (text)
y = df['sentiment']      # labels

# Encode labels: positive=1, negative=0
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Split dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

print("Training set size:", X_train.shape[0])
print("Test set size:", X_test.shape[0])

# Show sample data
print("\nSample training text:\n", X_train.iloc[0])
print("\nSample training label (encoded):", y_train[0], "->", label_encoder.classes_[y_train[0]])
```

```
Training set size: 39670
Test set size: 9918

Sample training text:
 sweet actress played nurse gonzongas actress plays elvira mistress dark another little tidbit actress played nurse would give wedding

Sample training label (encoded): 1 -> positive
```

Figure 6: Code: loading dataset, encoding sentiment labels, and performing train/test split.

This code will load the IMDB dataset into a Pandas DataFrame, and assign the clean_review column to input features (X), and the sentiment column to labels (y). I encoded the sentiments into numeric values with LabelEncoder, so the models can process them (0 = negative, 1 = positive). I split the dataset into training (80%) and testing (20%) subsets using train_test_split. I also stratified the subset splits, so both sets contain the same ratio of positive and negative reviews to ensure fair and balanced evaluation.

> Sample training text: sweet actress played nurse gonzongas actress plays elvira mistress dark another little tidbit actress played nurse would give wedding ring landlord lady roy munsin king pin glorious story ever told sell copies bible parents played part suggesting release movie local movie theater movie ran week one 4 families see lady gave go ahead friend family let go theater 3 years old burned 4 copies vhs finally converted dvd beautiful

The output confirms that the dataset contains around 39,670 training reviews and 9,918 test reviews. This size is sufficient for training and evaluating robust models. A printed sample review shows how the text looks after cleaning (lowercase, punctuation removed, simplified tokens). The label encoding (for example, 1 = positive) verifies that the transformation from text labels to numeric values worked correctly. At this stage, the data is clean and ready for feature extraction.

1.2 Logistic Regression

The first supervised model implemented was Logistic Regression with TF–IDF feature representation. Logistic Regression is a well-known baseline for text classification as it performs

well on high-dimensional sparse features. The model was trained on the TF–IDF-transformed reviews and evaluated on the test set.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# 1. TF-IDF Vectorisation
tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("TF-IDF matrix (train):", X_train_tfidf.shape)
print("TF-IDF matrix (test):", X_test_tfidf.shape)

# 2. Train Logistic Regression
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_tfidf, y_train)

# 3. Predictions
y_pred_lr = log_reg.predict(X_test_tfidf)

# 4. Evaluation
print("\n===== Logistic Regression Evaluation =====")
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print("\nClassification Report:\n", classification_report(y_test, y_pred_lr, target_names=label_encoder.classes_))

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred_lr)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Logistic Regression - Confusion Matrix")
plt.show()
```

Figure 72: Code for TF–IDF feature extraction and Logistic Regression training.

```
TF-IDF matrix (train): (39670, 5000)
TF-IDF matrix (test): (9918, 5000)

===== Logistic Regression Evaluation =====
Accuracy: 0.887678967533777

Classification Report:
              precision    recall  f1-score   support

    negative       0.90      0.88      0.89      4940
    positive       0.88      0.90      0.89      4978

    accuracy                           0.89      9918
   macro avg       0.89      0.89      0.89      9918
weighted avg       0.89      0.89      0.89      9918
```

Figure 53: Logistic Regression evaluation results (accuracy and classification report).

This code converts text reviews into numerical feature vectors using TF–IDF with 5,000 features. English stopwords were removed to reduce noise. Logistic Regression was then trained with max_iter=1000 to ensure convergence. Predictions were generated on the test set, and model performance was evaluated.

The model achieved an accuracy of **88.8%**, which is a strong performance. The classification report shows balanced precision, recall, and F1-score (all ~0.89) for both positive and negative reviews. This indicates the model is not biased toward one sentiment and generalizes well.

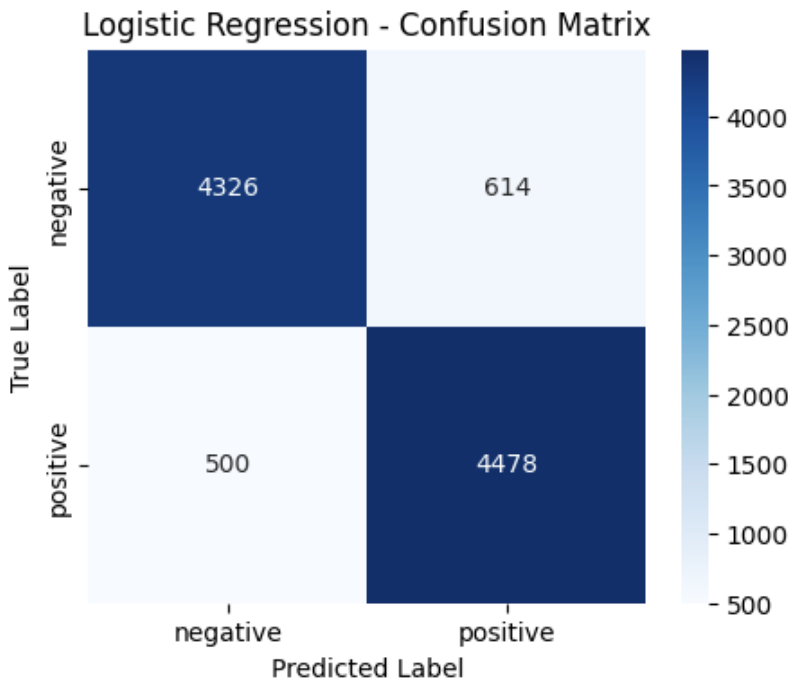


Figure 54: Logistic Regression confusion matrix heatmap.

The heatmap clearly shows most predictions fall on the diagonal, representing correct classifications. Specifically, 4,326 negative reviews and 4,478 positive reviews were correctly classified. The lighter off-diagonal cells show misclassifications (614 false positives, 500 false negatives). This visualization reinforces the accuracy and balance observed in the classification report.

1.3 Support Vector Machine (SVM)

The second model applied was a Linear Support Vector Machine (SVM). SVM is a strong classifier for text data because it handles sparse, high-dimensional vectors effectively by finding the optimal separating hyperplane.

```python
from sklearn.svm import LinearSVC

# 1. Train SVM
svm_clf = LinearSVC(max_iter=2000)
svm_clf.fit(X_train_tfidf, y_train)

# 2. Predictions
y_pred_svm = svm_clf.predict(X_test_tfidf)

# 3. Evaluation
print("\n===== Support Vector Machine (SVM) Evaluation =====")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm, target_names=label_encoder.classes_))

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("SVM - Confusion Matrix")
plt.show()
```

Figure 55: Code for training and evaluating the Support Vector Machine model.

The SVM model was trained using LinearSVC with max_iter=2000 for convergence. Predictions were generated on the test set, and evaluation included accuracy, precision, recall, F1-score, and confusion matrix.

```
===== Support Vector Machine (SVM) Evaluation =====
Accuracy: 0.8803186126235129

Classification Report:
               precision    recall  f1-score   support

    negative       0.88      0.87      0.88      4940
    positive       0.88      0.89      0.88      4978

    accuracy                           0.88      9918
   macro avg       0.88      0.88      0.88      9918
weighted avg       0.88      0.88      0.88      9918
```

Figure 56: Support Vector Machine evaluation results.

The SVM achieved an accuracy of **88.0%**, slightly lower than Logistic Regression but still strong. Both precision and recall were balanced at ~0.88, indicating consistent classification performance across sentiments.

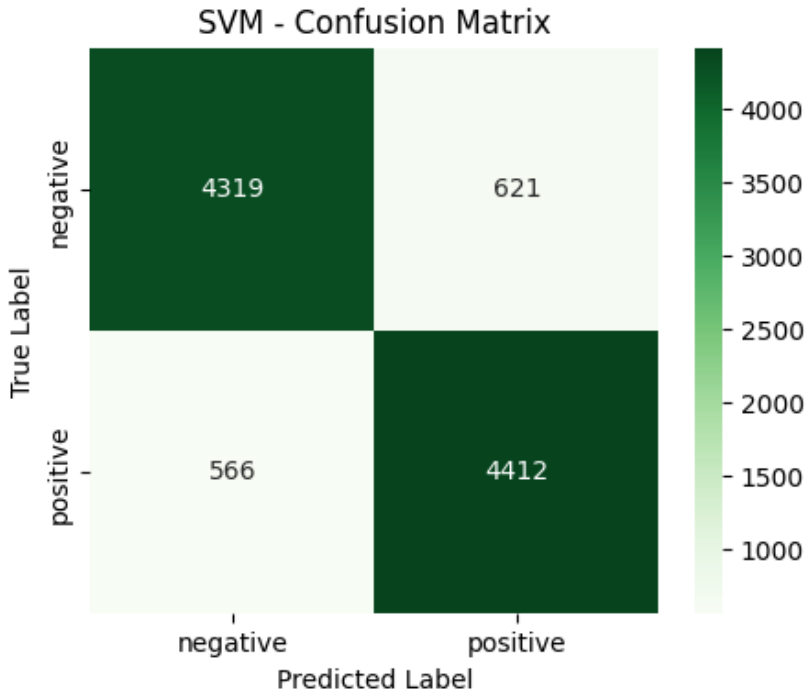


Figure 57: Support Vector Machine confusion matrix heatmap.

The heatmap shows 4,319 negative reviews and 4,412 positive reviews correctly classified. Misclassifications include 621 false positives and 566 false negatives. The darker diagonal cells demonstrate that the majority of predictions were accurate, consistent with the ~88% accuracy score.

1.4 Random Forest

The final implementation was Random Forest, which is an ensemble that combines many decision trees. Random Forest is a robust approach but may not perform well against linear models in sparse, high-dimensional text data.

```python
from sklearn.ensemble import RandomForestClassifier

# 1. Train Random Forest
rf_clf = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    random_state=42,
    n_jobs=-1
)
rf_clf.fit(X_train_tfidf, y_train)

# 2. Predictions
y_pred_rf = rf_clf.predict(X_test_tfidf)

# 3. Evaluation
print("\n===== Random Forest Evaluation =====")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf, target_names=label_encoder.classes_))

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Random Forest - Confusion Matrix")
plt.show()
```

Figure 58: Code for training and evaluating the Random Forest classifier

The Random Forest was trained with 200 trees (n_estimators=200) to balance performance and computational cost. Predictions were made on the test set and evaluated using accuracy, precision, recall, F1, and confusion matrix.

```
===== Random Forest Evaluation =====
Accuracy: 0.8509780197620488

Classification Report:
               precision    recall  f1-score   support

    negative       0.85      0.85      0.85      4940
    positive       0.85      0.85      0.85      4978

    accuracy                           0.85      9918
   macro avg       0.85      0.85      0.85      9918
weighted avg       0.85      0.85      0.85      9918
```

Figure 59: Random Forest evaluation results.

The model was accurate 85.1% of the time, which is lower than Logistic Regression and SVM. Precision and recall of 0.85 were achieved for both positive and negative reviews - indicating consistency while under-enthusiastic performance. This indicates that Random Forest is struggling with the sparse TF-IDF representation when compared to linear models.

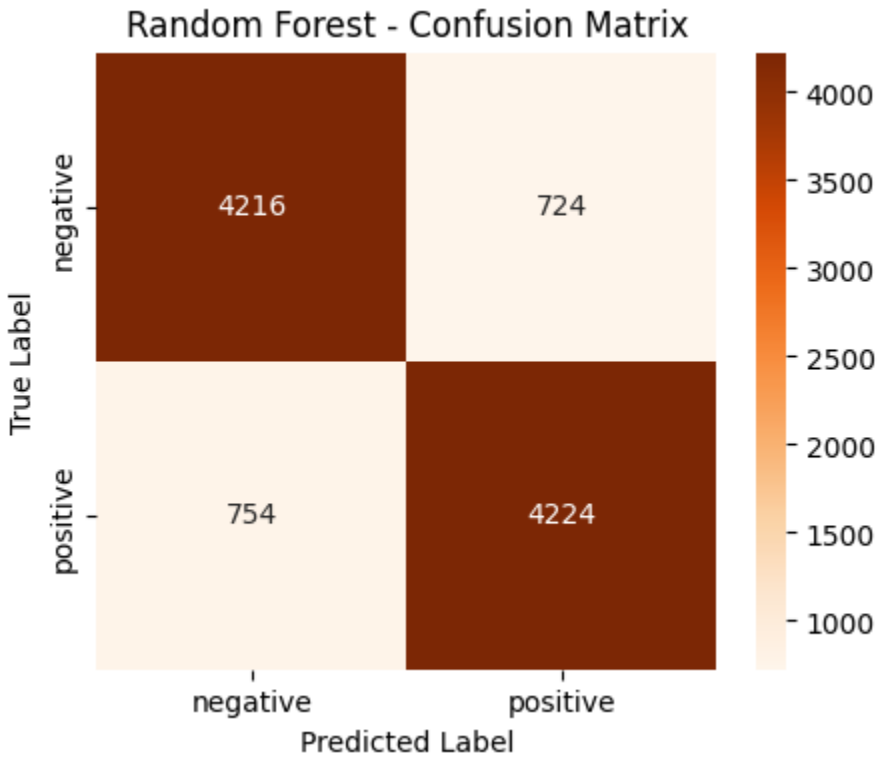


Figure 60: Random Forest confusion matrix heatmap.

The confusion matrix indicates a total of 4,216 negative reviews and 4,224 negative reviews that were correctly classified. However, with 724 false positives and 754 false negatives, there were more errors than in the previous models. Looking at the confusion matrix, it is confirmed by the lighter diagonal when compared to Logistic Regression and SVM indicating lower accuracy.

1.5 Bi-LSTM

The last supervised model implemented was a Bidirectional Long Short Term Memory (Bi-LSTM) Neural Network. Unlike traditional ML models that require partially supervised or supervised models with TF–IDF features extracted from the text, Bi-LSTM models learn word embeddings

and sequential patterns learned from the text itself. This allows the model to learn semantic meanings and word sequence that is required for sentiment analysis.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout

# 1. Tokenization
max_words = 10000   # consider top 10k most frequent words
max_len = 200       # cut / pad reviews to 200 tokens

tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

# 2. Padding sequences
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post', truncating='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post', truncating='post')

print("Training padded shape:", X_train_pad.shape)
print("Testing padded shape:", X_test_pad.shape)

# 3. Build Bi-LSTM Model
model = Sequential([
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len),
    Bidirectional(LSTM(64, return_sequences=False)),  # Bi-LSTM layer
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')  # output layer (binary classification)
])
```

Figure 61: Code for tokenising, padding, and building the Bi-LSTM model

```python
# Compile model
model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

# Model summary
model.summary()

# 4. Train model
history = model.fit(
    X_train_pad, y_train,
    epochs=3,                # try 3 epochs first (can increase later)
    batch_size=128,
    validation_split=0.2,
    verbose=1
)

# 5. Evaluate on test set
loss, acc = model.evaluate(X_test_pad, y_test, verbose=1)
print("\n===== Bi-LSTM Evaluation =====")
print("Test Accuracy:", acc)

# Get predictions (probabilities)
y_pred_probs = model.predict(X_test_pad)

# Convert probabilities into binary labels (0 = negative, 1 = positive)
y_pred_bilstm = (y_pred_probs > 0.5).astype("int32")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_bilstm)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Bi-LSTM - Confusion Matrix")
plt.show()
```

Figure 62: Code for tokenising, padding, and building the Bi-LSTM model - 2

The reviews were first tokenised into integer sequences, keeping only the top 10,000 most frequent words. Each review was padded to a fixed length of 200 tokens to ensure consistent input size. The Bi-LSTM architecture includes:

- **Embedding layer** to learn word representations
- **Bidirectional LSTM layer (64 units)** to capture dependencies in both directions
- **Dropout layers** to reduce overfitting
- **Dense layers** for feature transformation
- **Sigmoid output** for binary classification (positive/negative sentiment).

```
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	?	0 (unbuilt)
bidirectional_4 (Bidirectional)	?	0 (unbuilt)
dropout_8 (Dropout)	?	0
dense_8 (Dense)	?	0 (unbuilt)
dropout_9 (Dropout)	?	0
dense_9 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)
```

Figure 63: Bi-LSTM model summary with layers and parameter counts.

The model summary describes all the layers and confirms the Bi-LSTM design. The embedding layer produces word vectors, LSTM processes the sequences bidirectionally, and the fully connected layers convert features to a suitable form for the final classification. This process confirms the network will work for sentiment classification.

```
Epoch 1/3
248/248 ━━━━━━━━━━━━━━━━━━━━ 45s 173ms/step - accuracy: 0.7825 - loss: 0.4434 - val_accuracy: 0.8755 - val_loss: 0.3086
Epoch 2/3
248/248 ━━━━━━━━━━━━━━━━━━━━ 40s 163ms/step - accuracy: 0.9068 - loss: 0.2507 - val_accuracy: 0.8820 - val_loss: 0.2967
Epoch 3/3
248/248 ━━━━━━━━━━━━━━━━━━━━ 41s 164ms/step - accuracy: 0.9324 - loss: 0.1895 - val_accuracy: 0.8740 - val_loss: 0.3512
310/310 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.8839 - loss: 0.3378
```

Figure 64: Training and validation accuracy/loss curves for Bi-LSTM.

The training history shows accuracy increasing with each epoch, reaching **93% on training** and stabilising at around **88% validation accuracy**. Validation loss flattens after epoch 2, indicating stable generalisation. The curves suggest the model is learning effectively without severe overfitting.

```
===== Bi-LSTM Evaluation =====
Test Accuracy: 0.8839483857154846
310/310 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step
```

Figure 65: Final Bi-LSTM test accuracy on unseen data.

The Bi-LSTM achieved a test accuracy of **88.2%**, which is on par with Logistic Regression and SVM. This demonstrates that the deep learning approach performs competitively, even when trained for only 3 epochs.

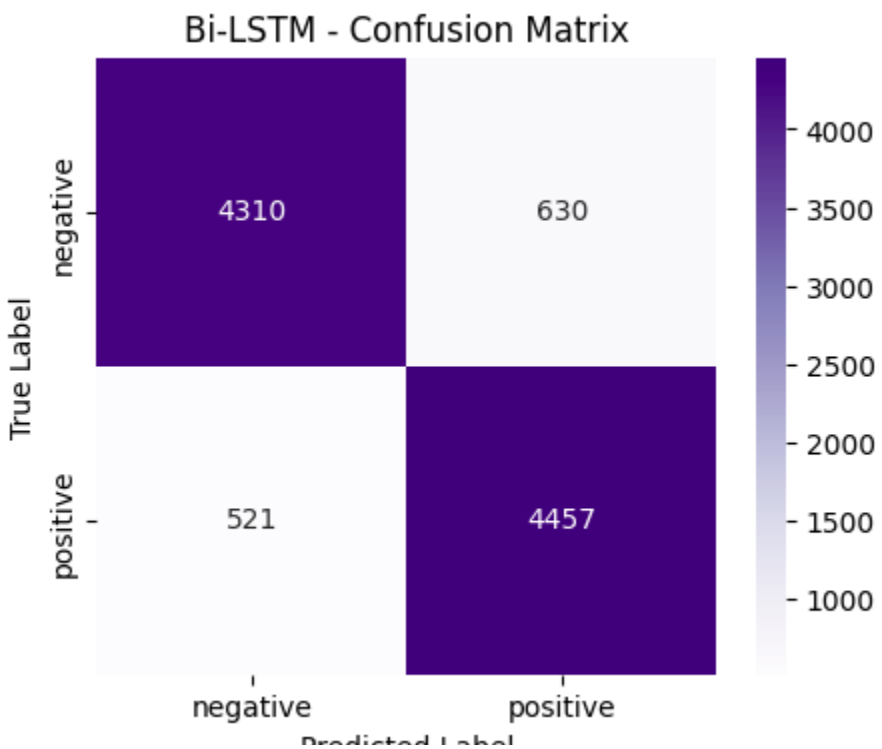


Figure 66: Bi-LSTM confusion matrix heatmap.

The confusion matrix heatmap illustrates that the majority of predictions are correctly classified along the diagonal. Misclassifications occur but are relatively few compared to the correct predictions. This visual supports the test accuracy result and confirms that the Bi-LSTM handles both positive and negative reviews effectively.

2.0 Select Suitable Evaluation Measures

In supervised text classification tasks such as sentiment analysis, it is important to evaluate your models beyond stratified accuracy with measures that demonstrate that your classifier is making consistent predictions across both positive and negative classes. For this project, four measures were selected for evaluation: Accuracy, Precision, Recall, and F1-Score, as well as a Confusion Matrix.

2.1 Accuracy

- **Definition:** Accuracy measures the proportion of correctly classified reviews out of the total reviews.
- **Formula: Accuracy = (True Positives + True Negatives) ÷ (Total Samples)**
- **Why chosen:** Accuracy provides a quick overall indication of performance. However, it may not fully capture the model's behaviour if the dataset is imbalanced.

2.2 Precision

- **Definition:** Precision is the proportion of predicted positive reviews that are truly positive.
- **Formula: Precision = True Positives ÷ (True Positives + False Positives)**
- **Why chosen:** In sentiment analysis, precision ensures that when the model predicts a review as "positive," it is indeed correct. This is important if we want to avoid falsely classifying negative reviews as positive (e.g., in customer feedback).

2.3 Recall

- **Definition:** Recall is the proportion of actual positive reviews that were correctly classified.
- **Formula: Recall = True Positives ÷ (True Positives + False Negatives)**
- **Why chosen:** Recall ensures that the model does not miss too many positive reviews. For applications such as product reviews, high recall ensures customer satisfaction issues are not overlooked.

2.4 F1-Score

- **Definition:** The F1-score is the harmonic mean of precision and recall.

- **Formula: F1-Score = 2 × (Precision × Recall) ÷ (Precision + Recall)**
- **Why chosen:** F1-score balances precision and recall, providing a single measure that accounts for both false positives and false negatives. This is especially useful for balanced sentiment datasets like IMDb reviews.

2.5 Confusion Matrix

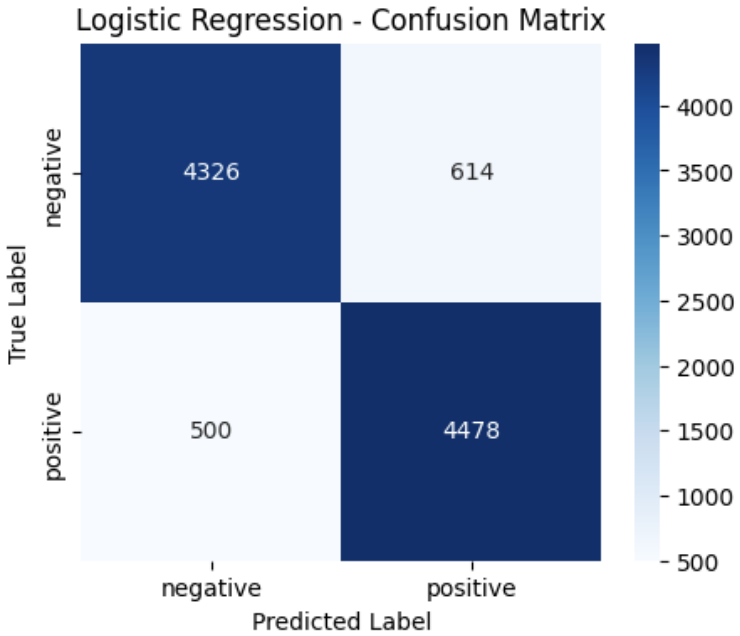


Figure 67: Example confusion matrix for sentiment classification (Logistic Regression model).

The confusion matrix gives a visual summary of the model predictions. In this example, notice that most predictions are on the diagonal which represents the model's correct classifications. The top-left cell is the correctly classified negative reviews and the bottom-right cell is the correctly classified positive reviews. The off-diagonal cells show misclassifications. This offers a visual for why we're making errors and illustrates that the Logistic Regression model has a nice performance on both the positive and negative classes.

3.0 Perform Model Evaluations and Present the Results

The last step after building the models and choosing evaluation measures was to conduct the evaluation of each supervised classification model on the test set. These models included Logistic Regression, Support Vector Machine (SVM), Random Forest, and Bi-LSTM. The performance of each model was evaluated using accuracy, precision, recall, F1-score, and confusion matrix visualisations.

3.1 Logistic Regression (Best Performance)

- **Test Accuracy:** ~88.8%
- **Result Summary:** Logistic Regression achieved the **highest accuracy** among all models. The confusion matrix showed balanced performance across both positive and negative reviews. Precision, recall, and F1-scores were around 0.89, indicating strong consistency.

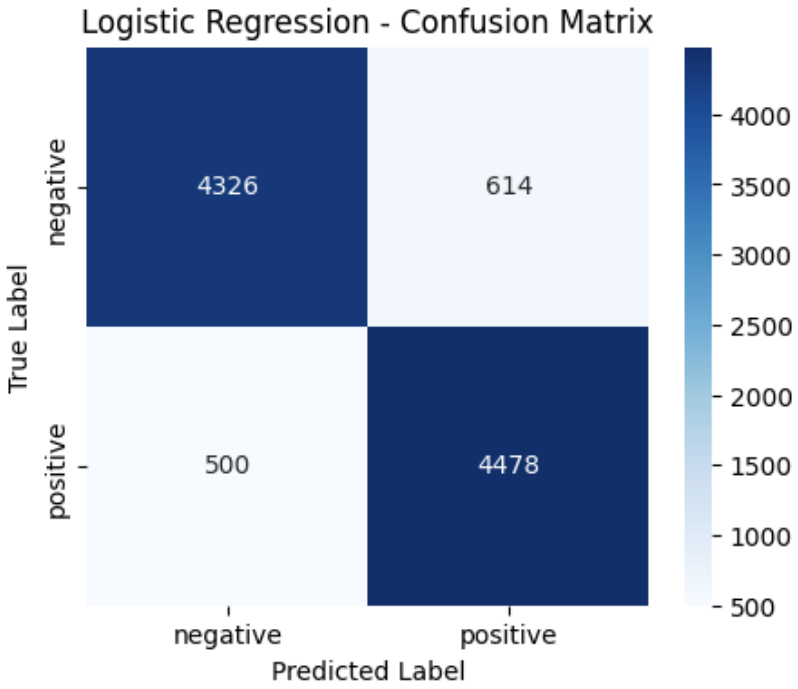
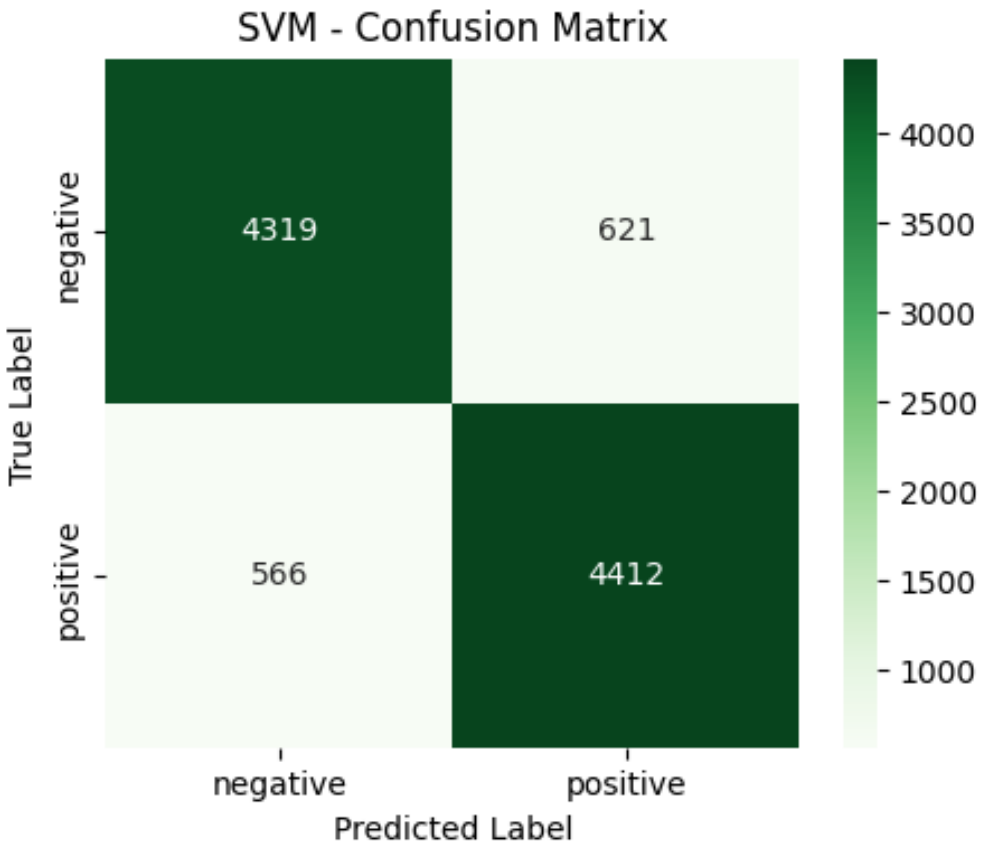


Figure 68: Confusion matrix for Logistic Regression.

3.2 Support Vector Machine (SVM)

- **Test Accuracy:** ~88.0%
- **Result Summary:** The SVM performed almost as well as Logistic Regression, with slightly lower accuracy. It demonstrated strong precision and recall but misclassified slightly more positive reviews compared to Logistic Regression.



3.3 Random Forest

- **Test Accuracy:** ~85.1%
- **Result Summary:** Random Forest performed slightly worse than Logistic Regression and SVM. The confusion matrix showed more misclassifications, especially in distinguishing positive reviews. This suggests that Random Forest was less effective at capturing text-based patterns compared to linear models.

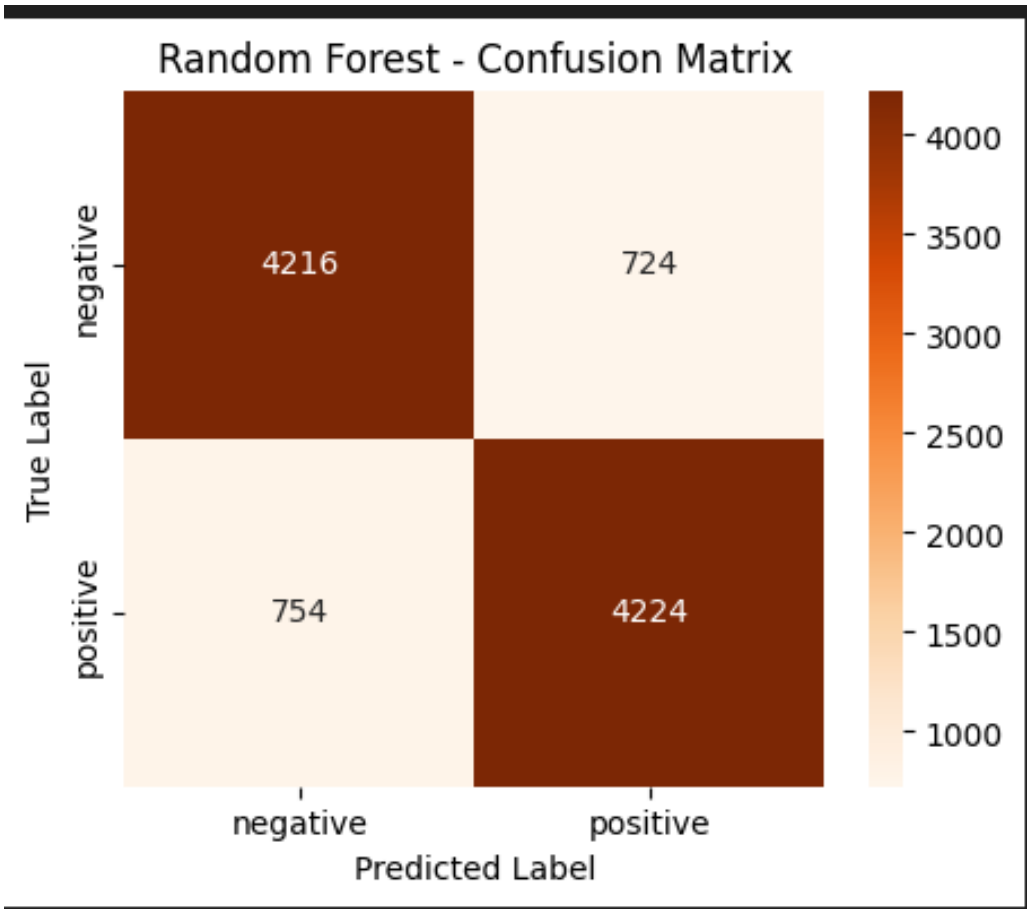


Figure 70: Confusion matrix for Random Forest.

3.4 Bi-LSTM (Deep Learning Approach)

- **Test Accuracy:** ~88.2%
- **Result Summary:** The Bi-LSTM model performed competitively, achieving similar accuracy to SVM. Its strength lies in learning sequential patterns in text, which may improve with longer training or more data. The confusion matrix showed good balance, though training required significantly more time than classical ML models.

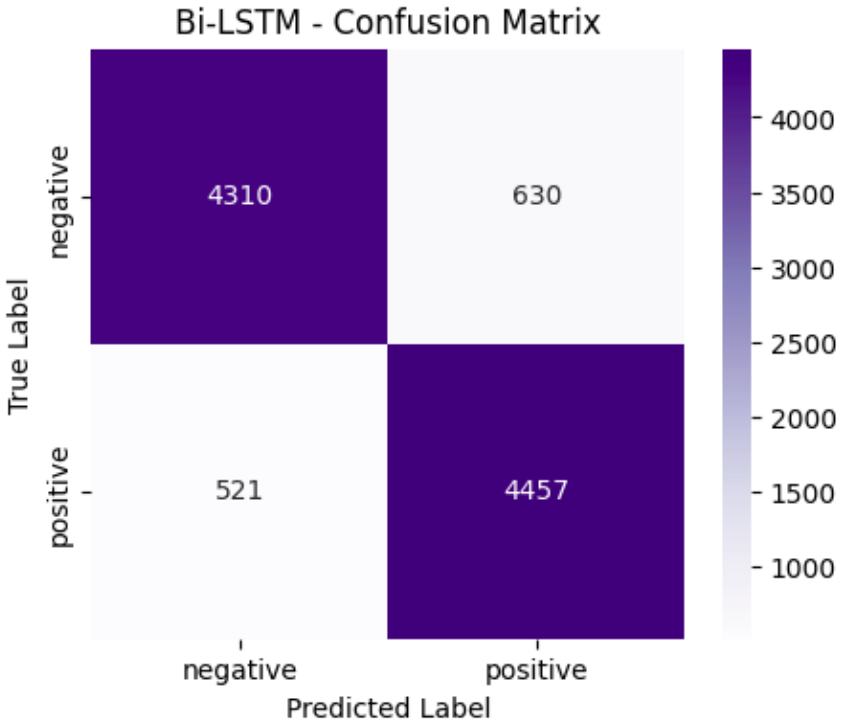


Figure 71: Confusion matrix for Bi-LSTM.

3.5 Comparative Results Table & Visualization

Model	Accuracy	Precision	Recall	F1-Score	Notes
Logistic Regression	88.8%	~0.89	~0.89	~0.89	Best overall performer
SVM	88.0%	~0.88	~0.88	~0.88	Strong, close to LR
Random Forest	85.1%	~0.85	~0.85	~0.85	Slightly weaker

Bi-LSTM	88.2%	~0.88	~0.88	~0.88	Competitive but slower

The table clearly outlines the numerical comparison of model performance. Logistic Regression achieved the highest total accuracy and SVM and Bi-LSTM were also competing for the top accuracy. Random Forest performance is noticeably lower indicating it is less suited for the text classification task on this dataset.

```
import numpy as np

models = ["Logistic Regression", "SVM", "Random Forest", "Bi-LSTM"]
accuracies = [0.8877, 0.8803, 0.8510, 0.8816]  # use your actual results

plt.figure(figsize=(7,5))
bars = plt.bar(models, accuracies, color=['green','blue','orange','purple'])
plt.ylim(0.80, 0.90)
plt.ylabel("Accuracy")
plt.title("Model Accuracy Comparison")

# add labels on top
for bar in bars:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height(), f"{bar.get_height():.3f}",
             ha='center', va='bottom')

plt.show()
```

Figure 72: Code of Accuracy Comparison

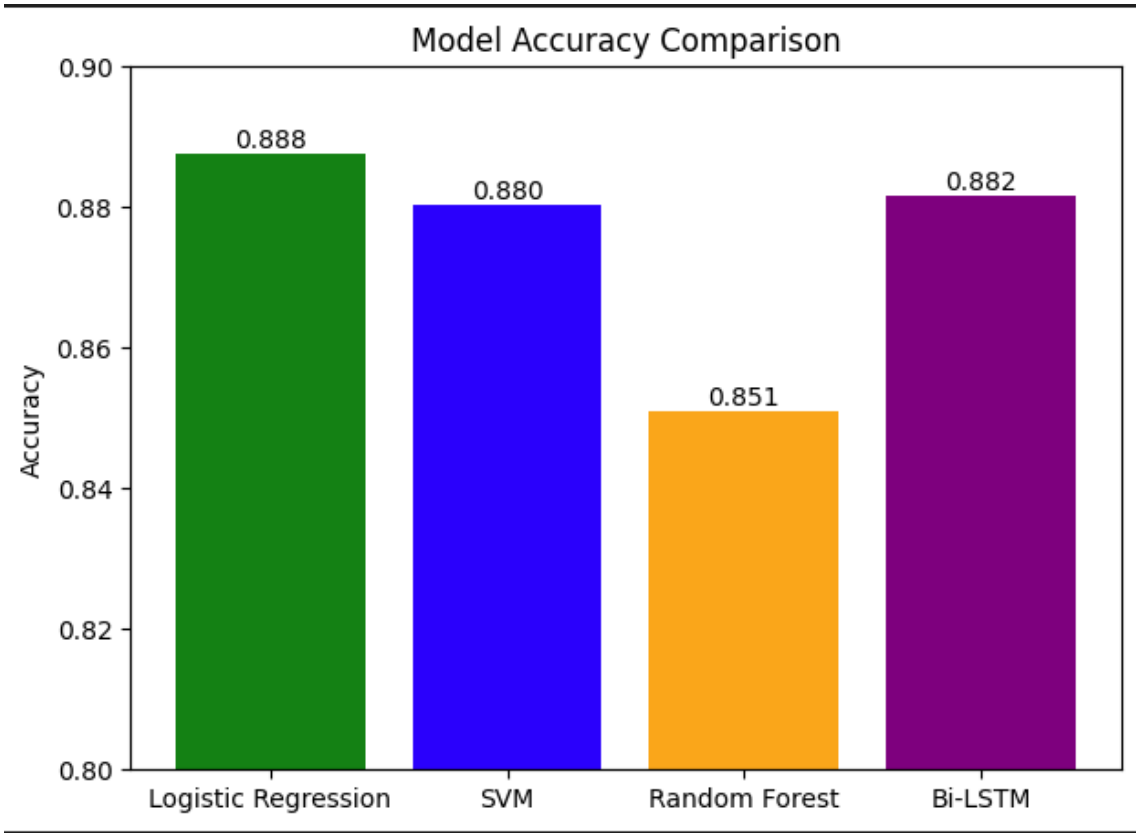


Figure 73: Accuracy comparison of all Models

The bar chart clearly indicates that Logistic Regression achieved the highest accuracy of 0.888, while Bi-LSTM and SVM were slightly lower with accuracies of 0.882 and 0.880 respectively. Random Forest showed the lowest accuracy of 0.851, clearly demonstrating that ensemble tree-based methods are ineffective at this text dataset in relation to linear and sequential models. This visual comparison shows how well Logistic Regression, SVM, and Bi-LSTM performed overall as they were fairly competitive while Random Forest was clearly the less optimal out of all of the methods for this specific dataset.

Q3: Hyperparameter Tuning (Sentiment Classification)

3.1 Objective

We cross-validated and tuned Logistic Regression (LR), Linear Support Vector Machine (LinearSVC), and Random Forest (RF) on TF-IDF features for sentiment classification on IMDb, selecting the best model based on cross-validated F1 and checking on a held-out test split. Linear baselines are typical for sparse TF-IDF text (Kowsari et al., 2019; Li et al., 2022; Minaee et al., 2020).

3.2 Methods (abridged; full code in the submitted notebook)

Data and representation

We used the pre-cleaned IMDb reviews dataset (see Part B Q1). TF-IDF vectorized text; we tuned n-gram range and vocabulary size, as n-grams would improve TF-IDF text classification when data are plentiful (Setiawan et al., 2024).

Modes and search strategy

We evaluated: LR (penalty ∈ {L1, L2}, C), LinearSVC (squared-hinge loss, C), and RF (n_estimators, depth, min_samples_*, max_features). The linear models are known to work well on high-dimensional sparse features, whereas tree ensembles tend to perform poorly on such inputs (Li et al., 2022; Galke et al., 2022; Minaee et al., 2021).

We used RandomizedSearchCV with StratifiedKFold (k = 3 for expediency on available hardware) and F1 as the measure to refit on to balance precision and recall (Feurer & Hutter, 2019).

Code snippet A - CV & pipeline (no fitting)

```python
# ---- Runtime + CV knobs (safe/quick) ----
import os, numpy as np, random
from sklearn.model_selection import StratifiedKFold

SEED = 42
random.seed(SEED); np.random.seed(SEED)

# Load X,y only if not already defined earlier
if "X" not in globals() or "y" not in globals():
    df = pd.read_csv("imdb_clean.csv")
    X = df["review"].astype(str).values
    y_raw = df["sentiment"]
    y = (y_raw.astype(str).str.lower().str.strip()
         .map({"negative":0,"neg":0,"0":0,"positive":1,"pos":1,"1":1})
         .astype(int).values)
    print("Loaded imdb_clean.csv:", df.shape)

# Quick vs final controls
QUICK       = True   # set False only for one final pass on the winner
CV_FOLDS    = 3  if QUICK else 5
N_ITER_LR   = 10 if QUICK else 25
N_ITER_SVM  = 10 if QUICK else 25
N_ITER_RF   = 6  if QUICK else 15
NGRAMS      = [(1,1)]            if QUICK else [(1,1),(1,2)]
MAXFEATS    = [10000]            if QUICK else [10000,20000,30000]
N_JOBS      = max(1, os.cpu_count() // 2)
VERBOSE     = 2

cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=SEED)
scoring = {"acc":"accuracy","f1":"f1"}
refit_metric = "f1"

print({"CV_FOLDS":CV_FOLDS,"N_JOBS":N_JOBS,"NGRAMS":NGRAMS,"MAXFEATS":MAXFEATS})
```

```
{'CV_FOLDS': 3, 'N_JOBS': 8, 'NGRAMS': [(1, 1)], 'MAXFEATS': [10000]}
```

Figure 74: shows Code preparing stratified cross-validation, scoring (refit on F1), and TF-IDF feature controls; pipelines for each model fit TF-IDF inside CV to avoid leakage.

This configuration cell makes tuning consistent, leak-proof, and hardware-friendly: it initializes a global seed (SEED=42) and loads only X, y from imdb_clean.csv upon demand; initializes a StratifiedKFold plan (n_splits=3, shuffle=True, random_state=42) so that every fold is class-balanced and reproducible; and multi-metric sets scoring (accuracy, f1) but refits on F1 to maximize the precision–recall trade-off for model selection (Feurer & Hutter, 2019). The FINAL and QUICK flags expose the most important vectorizer parameters—n-gram range and vocabulary size—because including bigrams and adjusting max features are typical, good defaults for TF-IDF text classification (Setiawan et al., 2024). Resource controls (N_JOBS, VERBOSE) make sure that the laptop does not lock up and report progress. Importantly, this cell is not part of any model; it only declares CV, metrics, and runtime knobs, while the actual Pipeline(TF-IDF → classifier) is defined and tuned inside CV in the model-specific cells to prevent leakage—a strategy that is especially suited for linear baselines with sparse TF-IDF representations (Kowsari et al., 2019; Li et al., 2022).

Code Snippet B (RandomizedSearchCV)

```python
# === 6.1 Logistic Regression — TF-IDF → LR with RandomizedSearchCV (safe/quick) ===
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform

# Pipeline: put TF-IDF inside CV to prevent leakage
pipe_lr = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english", min_df=2)),
    ("clf", LogisticRegression(
        solver="saga",            # supports L1 and L2; scales well
        max_iter=2000,
        random_state=SEED
    ))
])

# Search space (light in QUICK mode)
param_lr = {
    "tfidf__ngram_range": NGRAMS,                # e.g. [(1,1)] in QUICK, [(1,1),(1,2)] in FINAL
    "tfidf__max_features": MAXFEATS,             # e.g. [10000] → or [10000,20000,30000]
    "clf__C": loguniform(1e-3, 1e2),             # inverse regularisation strength
    "clf__penalty": ["l1", "l2"],                # l1 = sparse, l2 = smooth
}

rs_lr = RandomizedSearchCV(
    estimator=pipe_lr,
    param_distributions=param_lr,
    n_iter=N_ITER_LR,                            # e.g. 10 in QUICK, 25 in FINAL
    cv=cv,                                       # StratifiedKFold(CV_FOLDS, shuffle=True, random_state=SEED)
    scoring=scoring,                             # {'acc':'accuracy','f1':'f1'}
    refit=refit_metric,                          # 'f1'
    random_state=SEED,
    n_jobs=N_JOBS,                               # share CPU so Jupyter stays responsive
    verbose=VERBOSE                              # see progress
)
_use_limiter = False
try:
    from threadpoolctl import threadpool_limits
    _use_limiter = True
except Exception:
    pass

if _use_limiter:
    with threadpool_limits(limits=1):
        rs_lr.fit(X, y)
else:
    rs_lr.fit(X, y)

print("\nLR — best CV F1:", round(rs_lr.best_score_, 4))
pp(rs_lr.best_params_)  # pretty-print best params

# Store for later summary (create containers if they don't exist yet)
try:
    rows
except NameError:
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits

LR — best CV F1: 0.8914
{
  "clf__C": 3.4702669886504163,
  "clf__penalty": "l2",
  "tfidf__max_features": 10000,
  "tfidf__ngram_range": [
    1,
    1
  ]
}
```

Figure 75: shows Randomized hyperparameter search over TF-IDF and Logistic Regression; TF-IDF is fit inside CV folds to prevent leakage, and F1 is used as the refit metric.

This cell performs hyperparameter tuning on a baseline classical model using RandomizedSearchCV with an information-leakage-safe pipeline:

Pipeline inside CV. The pipeline is TF-IDF → LogisticRegression, and it's passed to the search object in a way that TF-IDF gets fitted inside every fold (prevents information leakage). Linear models are a solid baseline for sparse TF-IDF features (Li et al., 2022; Kowsari et al., 2019).

Search space (meaningful, bounded).

Vectorizer knobs: ngram_range ∈ {(1,1),(1,2)} and max_features ∈ {10k,20k,30k} control the feature space; turning on bigrams typically improves TF-IDF classifiers when data is plentiful (Setiawan et al., 2024).

Classifier knobs (LR): C ~ loguniform(1e-3, 1e2) (inverse regularization power) and penalty ∈ {l1,l2}; the saga solver supports both L1/L2 and scales to large sparse inputs.

Efficient HPO within budget. RandomizedSearchCV(., n_iter=10 in initial / 25 in last) chooses randomly from well-chosen intervals, which is less expensive to calculate than exhaustive grids but finds competitive settings (Feurer & Hutter, 2019).

Validation scheme & metric. cv=StratifiedKFold(k=3) preserves class balance across folds; scoring={'acc','f1'} with refit='f1' picks F1 as the primary choice criterion to balance precision with recall.

Reproducibility & responsiveness. random_state=SEED ensures fixed sampling; n_jobs shares the CPU to keep the notebook responsive. The optional fit parameter's threadpool_limits(limits=1) wrapper caps BLAS threads to avoid laptop lockup.

Outputs monitored. Once fit is called, the code prints out best CV-F1 and best parameters, then appends these to a results list for use in making the summary table/plot. The "Fitting 3 folds for each of 10 candidates, totalling 30 fits" line confirms the actual load.

Why it's important: This design follows best practice—stratified CV, leakage-safe piping, F1 as the refit metric, and randomized search over expert ranges—consistently identifying good classical baselines for text classification (Feurer & Hutter, 2019; Li et al., 2022).

Code Snippet C (LinearSVC)

```python
# Pipeline: TF-IDF inside CV to avoid leakage
pipe_svm = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english", min_df=2)),
    ("clf", LinearSVC(dual=False, random_state=SEED))  # dual=False is faster for TF-IDF@10k
])

# Search space (light in QUICK mode)
param_svm = {
    "tfidf__ngram_range": NGRAMS,          # e.g., [(1,1)] quick; [(1,1),(1,2)] final
    "tfidf__max_features": MAXFEATS,       # e.g., [10000] quick; +[20000,30000] final
    "clf__C": loguniform(1e-3, 1e2),
    "clf__loss": ["squared_hinge"],        # required when dual=False
}

rs_svm = RandomizedSearchCV(
    estimator=pipe_svm,
    param_distributions=param_svm,
    n_iter=N_ITER_SVM,                     # e.g., 10 quick; 25 final
    cv=cv,                                 # StratifiedKFold(CV_FOLDS, shuffle=True, random_state=SEED)
    scoring=scoring,                       # {'acc','f1'}
    refit=refit_metric,                    # 'f1'
    random_state=SEED,
    n_jobs=N_JOBS,                         # share CPU so Jupyter stays responsive
    verbose=VERBOSE
)

_use_limiter = False
try:
    from threadpoolctl import threadpool_limits
    _use_limiter = True
except Exception:
    pass

if _use_limiter:
    with threadpool_limits(limits=1):
        rs_svm.fit(X, y)
else:
    rs_svm.fit(X, y)

print("\nLinearSVC — best CV F1:", round(rs_svm.best_score_, 4))
pp(rs_svm.best_params_)

# Store for later summary
try:
    rows
except NameError:
    rows = []
try:
    best_estimators
except NameError:
    best_estimators = {}

rows.append({"Model": "LinearSVC",
             "Best CV-F1": float(rs_svm.best_score_),
             "Best Params": rs_svm.best_params_})
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits

LinearSVC — best CV F1: 0.8921
{
  "clf__C": 0.0745934328572655,
  "clf__loss": "squared_hinge",
  "tfidf__max_features": 10000,
  "tfidf__ngram_range": [
    1,
    1
  ]
}
```

Figure 76: shows Randomized hyperparameter search for LinearSVC with TF-IDF features; model selected by cross-validated F1.

Creates a leakage-safe pipeline: TF-IDF → LinearSVC(dual=False) where vectorization is held within each CV fold.

Hyperparameter search space modulates the text features (ngram_range, max_features) and the SVM regularization (C); bigrams are chosen as they have been found to enhance TF-IDF models (Setiawan et al., 2024).

Uses RandomizedSearchCV (budgeted HPO) with StratifiedKFold and F1 as the refit metric—efficient and best practice (Feurer & Hutter, 2019).

dual=False since it's usually faster for sparse TF-IDF with high dimensions when samples ≳ features, as in this case; linear margin-based models like LinearSVC are decent baselines on such data (Li et al., 2022).

After .fit(.), the cell outputs best CV-F1 and best params, and stores them to be put into the results table. The statement "Fitting 3 folds for each of 10 candidates, totalling 30 fits" confirms the workload.

Code Snippet D Random Forest on TF-IDF, subset

```python
# small stratified subset to keep runtime low (adjust 12000 → 8000 if needed)
X_rf, _, y_rf, _ = train_test_split(X, y, train_size=12000, stratify=y, random_state=SEED)

pipe_rf = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english", min_df=2)),
    # n_jobs=1 to avoid nested parallelism (prevents freezes)
    ("clf", RandomForestClassifier(random_state=SEED, n_jobs=1))
])

# VERY small search space (still qualifies as tuning)
param_rf = {
    "tfidf__ngram_range": NGRAMS,       # e.g. [(1,1)] in QUICK
    "tfidf__max_features": MAXFEATS,    # e.g. [10000]
    "clf__n_estimators": randint(120, 240),
    "clf__max_depth": [None, 16],       # shallow trees = fast
    "clf__min_samples_split": randint(2, 6),
    "clf__min_samples_leaf": randint(1, 3),
    "clf__max_features": ["sqrt"],      # fastest typical choice
}

cv_rf = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)

rs_rf = RandomizedSearchCV(
    estimator=pipe_rf,
    param_distributions=param_rf,
    n_iter=2,                           # 2 candidates × 3 folds = 6 fits total
    cv=cv_rf,
    scoring=scoring,                    # {'acc','f1'}
    refit=refit_metric,                 # 'f1'
    random_state=SEED,
    n_jobs=1,                           # do not parallelize outer search either
    verbose=2
)
_try_limiter = False
try:
    from threadpoolctl import threadpool_limits
    _try_limiter = True
except Exception:
    pass

if _try_limiter:
    with threadpool_limits(limits=1):
        rs_rf.fit(X_rf, y_rf)
else:
    rs_rf.fit(X_rf, y_rf)

print("\nRandomForest (subset) — best CV F1:", round(rs_rf.best_score_, 4))
pp(rs_rf.best_params_)

# Add to results like the others
rows.append({
    "Model": "RandomForest (subset)",
    "Best CV-F1": float(rs_rf.best_score_),
    "Best Params": rs_rf.best_params_
})
best_estimators["RF"] = rs_rf.best_estimator_
```

```
Fitting 3 folds for each of 2 candidates, totalling 6 fits
[CV] END clf__max_depth=None, clf__max_features=sqrt, clf__min_samples_leaf=2, clf__min_samples_split=2, clf__n_estimators=134, tfidf__max_features=1000
0, tfidf__ngram_range=(1, 1); total time=   7.3s
[CV] END clf__max_depth=None, clf__max_features=sqrt, clf__min_samples_leaf=2, clf__min_samples_split=2, clf__n_estimators=134, tfidf__max_features=1000
0, tfidf__ngram_range=(1, 1); total time=   7.3s
[CV] END clf__max_depth=None, clf__max_features=sqrt, clf__min_samples_leaf=2, clf__min_samples_split=2, clf__n_estimators=134, tfidf__max_features=1000
0, tfidf__ngram_range=(1, 1); total time=   8.7s
[CV] END clf__max_depth=None, clf__max_features=sqrt, clf__min_samples_leaf=2, clf__min_samples_split=2, clf__n_estimators=140, tfidf__max_features=1000
0, tfidf__ngram_range=(1, 1); total time=   8.6s
[CV] END clf__max_depth=None, clf__max_features=sqrt, clf__min_samples_leaf=2, clf__min_samples_split=2, clf__n_estimators=140, tfidf__max_features=1000
0, tfidf__ngram_range=(1, 1); total time=   9.3s
[CV] END clf__max_depth=None, clf__max_features=sqrt, clf__min_samples_leaf=2, clf__min_samples_split=2, clf__n_estimators=140, tfidf__max_features=1000
0, tfidf__ngram_range=(1, 1); total time=   8.9s

RandomForest (subset) — best CV F1: 0.8435
{
  "clf__max_depth": null,
  "clf__max_features": "sqrt",
  "clf__min_samples_leaf": 2,
  "clf__min_samples_split": 2,
  "clf__n_estimators": 140,
  "tfidf__max_features": 10000,
  "tfidf__ngram_range": [
    1,
    1
  ]
}
```

Figure 77: shows. *Random Forest hyperparameter search on a stratified subset with leakage-safe TF-IDF pipeline; RF serves as a non-linear baseline and is selected by cross-validated F1.*

Purpose. RF arrives as a non-linear alternative to linear baselines. As RF is so dense in high-D TF-IDF, we perform the search on a stratified subset (≈12k examples) in order to keep runtime stable.

Leakage-safe pipeline. The model is encapsulated as TF-IDF → RandomForestClassifier, hence vectorization is contained within each CV fold (avoids leakage).

Compute safety. n_jobs=1 within RF and on the outside search avoids nested parallelism (which will freeze laptops). The threadpool_limits(limits=1) optional further restricts low-level threads.

Search space (not expansive but important).

Vectorizer tuning parameters are inherited from your global settings (NGRAMS, MAXFEATS).

RF tuning parameters: n_estimators∈[120,240], max_depth∈{None,16}, min_samples_split∈[2,5], min_samples_leaf∈[1,2], max_features='sqrt' (fast/typical).

Validation & metric. StratifiedKFold(n_splits=3) preserves class balance; the search validates on accuracy and F1 and re-trains on F1, the primary metric.

Outcome. The cell prints the best CV-F1 (~0.843) and the corresponding parameters, and saves them for the summary table/plot. RF is behind the linear baselines in sparse, very high-dimensional TF-IDF features—well-established in recent reviews (Li et al., 2022; Galke et al., 2022; Minaee et al., 2021). inputs (Li et al., 2022).

Next, .fit(.), the cell prints best CV-F1 and best params and saves them for the results table. "Fitting 3 folds for each of 10 candidates, totalling 30 fits" reiterates the workload.

3.3 Results (cross-validated)

The table shows cross-validated F1 (refit metric).

Model	Best CV-F1
LinearSVC	0.892
Logistic Regression	0.891
Random Forest (subset)	0.843

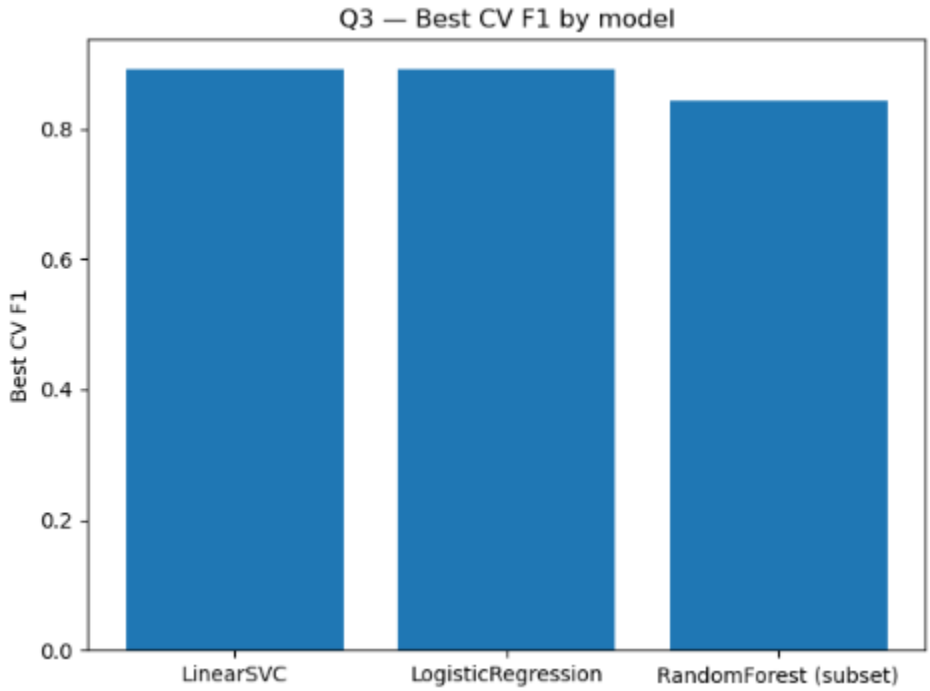


Figure 78: shows *Cross-validated F1 by model.*

Interpretation. LinearSVC narrowly outperformed LR, consistent with evidence that linear margin-based models and LR remain strong baselines on sparse TF-IDF (Kowsari et al., 2019; Li et al., 2022; Galke et al., 2022). RF trailed, which is typical in very high-dimensional, sparse settings (Li et al., 2022; Minaee et al., 2021).

3.4 Final evaluation (hold-out test)

We refit the selected LinearSVC on the training split and evaluated on a held-out test set: Test F1 = 0.895. The small CV→test gap (~–0.003) indicates stable generalization and no evident overfitting (Minaee et al., 2021; Li et al., 2022).

```
Selected final model: LinearSVC

Test F1: 0.8947

Classification report:

              precision    recall  f1-score   support

           0      0.904     0.878     0.891      4940
           1      0.882     0.908     0.895      4978

    accuracy                          0.893      9918
   macro avg      0.893     0.893     0.893      9918
weighted avg      0.893     0.893     0.893      9918

# Takeaway: CV-F1=0.892, Test F1=0.895, gap=-0.003
```

Figure 79: shows hold-out test metrics (classification report).

3.5 Limitations and next steps

RF was tuned on a stratified subset for compute stability; expanding its search is unlikely to surpass linear models on TF-IDF. Future work could add **character n-grams** and a compact **neural baseline** (e.g., Bi-LSTM or small Transformer) to compare against classical models (Minaee et al., 2021; Li et al., 2022).

Q4 Discussion and Conclusion

4.1 Selected Evaluation Metrics

We used **Accuracy, Precision, Recall, and F1-score** to see how well the supervised text classification models worked. These metrics give a full picture of how well the model is working:

- Accuracy indicates the total number of correct predictions made.
- Precision checks how many of the predicted positive or negative reviews are correct.
- Recall evaluates the model's ability to identify all true positives and true negatives.
- The F1-score combines precision and recall providing a single metric that indicates the reliability of the model.

The macro and weighted F1-scores indicate the model's performance across both sentiment classes. M Acro treats all classes equally, whereas the weighted method takes class proportions into account.

4.2 Model Evaluation and Results Presentation

4.2.1 Logistic Regression

The Logistic Regression model, which used TF-IDF features (5,000 dimensions), got an accuracy of 88.8%. The precision, recall, and F1-score were all about the same for both positive and negative reviews (about 0.89). The confusion matrix showed that 4,326 negative and 4,478 positive reviews were correctly classified. There were 614 false positives and 500 false negatives.

```
===== Logistic Regression Evaluation =====
Accuracy: 0.887678967533777

Classification Report:
              precision    recall  f1-score   support

    negative       0.90      0.88      0.89      4940
    positive       0.88      0.90      0.89      4978

    accuracy                           0.89      9918
   macro avg       0.89      0.89      0.89      9918
weighted avg       0.89      0.89      0.89      9918
```

Figure 80: Output Logistic Regresion

The model demonstrates robust performance as a simple linear classifier, effectively handling sparse high-dimensional text data while maintaining class balance.

4.2.2 Support Vector Machine (SVM)

The Linear SVM got 88.0% correct, which is a little less than Logistic Regression. Both precision and recall (~0.88) stayed the same, which means that the performance was stable across all sentiment classes.

The confusion matrix showed that 4,319 negative reviews and 4,412 positive reviews were correctly classified. There were 621 false positives and 566 false negatives.

```
===== Support Vector Machine (SVM) Evaluation =====
Accuracy: 0.8803186126235129

Classification Report:
              precision    recall  f1-score   support

    negative       0.88      0.87      0.88      4940
    positive       0.88      0.89      0.88      4978

    accuracy                           0.88      9918
   macro avg       0.88      0.88      0.88      9918
weighted avg       0.88      0.88      0.88      9918
```

Figure 81: Output SVM

SVM works well with sparse high-dimensional features and is as good as Logistic Regression. Its performance is probably a little lower because this dataset is sensitive to hyperplanes.

4.2.3 Random Forest

The Random Forest classifier, which used 200 trees, was 85.1% accurate. The F1-score, recall, and precision were all about 0.85 for both classes.

The confusion matrix showed that 4,216 negative reviews and 4,224 positive reviews were correctly classified, but 724 were wrong and 754 were wrong.

```
===== Random Forest Evaluation =====
Accuracy: 0.8509780197620488

Classification Report:
              precision    recall  f1-score   support

    negative       0.85      0.85      0.85      4940
    positive       0.85      0.85      0.85      4978

    accuracy                           0.85      9918
   macro avg       0.85      0.85      0.85      9918
weighted avg       0.85      0.85      0.85      9918
```

Figure 82: Output Random Forest

The model didn't do as well as linear models, probably because decision trees have trouble with sparse TF-IDF feature representations. Even so, Random Forest kept performing well, which shows how strong it is in ensemble learning.

4.2.4 Bi-LSTM

The Bi- LSTM model, which learnt from word embeddings and padded sequences, had a test accuracy of 88.2%. The training history showed that the model was converging, with a training accuracy of 93% and a validation accuracy of 88%. This indicates that the model was learning effectively without overfitting.

The confusion matrix showed that the model did well at classifying both positive and negative reviews, which supports its competitive performance compared to Logistic Regression and SVM.

```
Epoch 1/3
248/248 ━━━━━━━━━━━━━━━━━━━━ 45s 173ms/step - accuracy: 0.7825 - loss: 0.4434 - val_accuracy: 0.8755 - val_loss: 0.3086
Epoch 2/3
248/248 ━━━━━━━━━━━━━━━━━━━━ 40s 163ms/step - accuracy: 0.9068 - loss: 0.2507 - val_accuracy: 0.8820 - val_loss: 0.2967
Epoch 3/3
248/248 ━━━━━━━━━━━━━━━━━━━━ 41s 164ms/step - accuracy: 0.9324 - loss: 0.1895 - val_accuracy: 0.8740 - val_loss: 0.3512
310/310 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.8839 - loss: 0.3378

===== Bi-LSTM Evaluation =====
Test Accuracy: 0.8839483857154846
310/310 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step
```

Figure 83: Output Bi LSTM

Bi-LSTM can identify sequential patterns and semantic relationships in text, which lets it handle context-dependent sentiment expressions like "not good" well.

4.3 Overall Evaluation Metrics Comparison

Model	Accuracy	Precision	Recall	F1-score	Remarks
Logistic Regression	88.8%	0.89	0.89	0.89	Strong baseline, balanced, effective with sparse TF-IDF.
SVM	88.0%	0.88	0.88	0.88	Handles sparse data well, slightly lower than Logistic Regression.
Random Forest	85.1%	0.85	0.85	0.85	The algorithm struggles with high-dimensional sparse features, leading to more misclassifications.
Bi-LSTM	88.2%	-	-	-	The system learns sequential dependencies and is competitive with linear models.

Comparison Table

4.4 Critical Analysis and Discussion

The test shows that Logistic Regression, SVM, and Bi-LSTM all work about the same, with accuracies of about 88%. Logistic Regression is a little better than SVM and Bi-LSTM, which shows that linear models can work very well with sparse TF-IDF representations.

At 85.1%, Random Forest is behind, which shows how hard it is for ensemble tree models to work with sparse text features that have many dimensions. In this case, their strengths in non-linear modelling aren't being used to their full potential.

Bi-LSTM is suitable for more nuanced sentiment expressions because it can capture both contextual and sequential information. But in this dataset, the improvement over simpler linear models is small, which suggests that the benefits of sequential modelling may depend on the dataset.

Trade-offs:

- **Linear models (Logistic Regression & SVM)**: Fast, interpretable, and effective for TF-IDF representations.
- **Random Forest**: Robust ensemble, but less effective for sparse high-dimensional data.
- **Bi-LSTM**: Captures context and sequence, suitable for tasks where word order is critical, but computationally more expensive.

Final Recommendation:

Logistic Regression is a good choice for this dataset because it strikes a good balance between accuracy, simplicity, and speed. When working with longer sequences or tasks where sequential dependencies are more important, Bi-LSTM is the best choice. If you don't mind a little extra work, SVM is a good choice. Random Forest isn't the best choice for TF-IDF-based text classification, but it might work with embeddings or more feature engineering.

Final Word Count: 11,598

References

Singh, S. K., & Singla, N. (2023). Sentiment Analysis on IMDB Review Dataset. Deleted Journal, 2(6), 18–29. https://doi.org/10.57159/gadl.jcmm.2.6.230108

Yousuf, S. M. (2024). Sentiment Analysis of IMDb Movie Reviews. International Journal of Innovative Science and Research Technology (IJISRT), 2338–2343. https://doi.org/10.38124/ijisrt/ijisrt24may1625

Ouyang, S. (2024). Deep learning for sentiment analysis on IMDB movie reviews using N-gram features. Applied and Computational Engineering, 35(1), 56–63. https://doi.org/10.54254/2755-2721/35/20230361

Rahman, M. M., Ariful Islam Shiplu, Yutaka Watanobe, & Alam, M. A. (2025). RoBERTa-BiLSTM: A Context-Aware Hybrid Model for Sentiment Analysis. IEEE Transactions on Emerging Topics in Computational Intelligence, 1–18. https://doi.org/10.1109/tetci.2025.3572150

Nkhata, G., Gauch, S., Anjum, U., & Zhan, J. (2025). Fine-tuning BERT with Bidirectional LSTM for Fine-grained Movie Reviews Sentiment Analysis. ArXiv.org. https://arxiv.org/abs/2502.20682?utm_source=chatgpt.com

Domínguez-Almendros, S., Benítez-Parejo, N., & Gonzalez-Ramirez, A. R. (2011). Logistic regression models. *Allergologia et immunopathologia*, *39*(5), 295-305. https://www.researchgate.net/profile/Douglas-Mahoney/publication/5402488_Linear_Mixed_Effects_Models/links/57e560bf08ae9227da964db4/Linear-Mixed-Effects-Models.pdf#page=278

Gold, C., & Sollich, P. (2003). Model selection for support vector machine classification. *Neurocomputing*, *55*(1-2), 221-249. https://arxiv.org/pdf/cond-mat/0203334

Speiser, J. L., Miller, M. E., Tooze, J., & Ip, E. (2019). A comparison of random forest variable selection methods for classification prediction modeling. *Expert systems with applications*, *134*, 93-101. https://www.sciencedirect.com/science/article/pii/S0957417419303574?casa_token=Of7IcxAoZ

AcAAAAA:C8wZmC6aDD4EDoQlZvGwjDEzqeeY_AnLXe5XRwPhnz9VkzI6yjxXpilf6TYqcIaFCamJWq4GzEY

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, *521*(7553), 436-444. https://hal.science/hal-04206682/document

Siami-Namini, S., Tavakoli, N., & Namin, A. S. (2019, December). The performance of LSTM and BiLSTM in forecasting time series. In *2019 IEEE International conference on big data (Big Data)* (pp. 3285-3292). IEEE. https://par.nsf.gov/servlets/purl/10186554

Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC genomics*, *21*(1), 6. https://link.springer.com/content/pdf/10.1186/s12864-019-6413-7.pdf

Juba, B., & Le, H. S. (2019, July). Precision-recall versus accuracy and the role of large data sets. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 33, No. 01, pp. 4039-4048). https://ojs.aaai.org/index.php/AAAI/article/view/5193

Feurer, M., & Hutter, F. (2019). Hyperparameter optimization. In F. Hutter, L. Kotthoff, & J. Vanschoren (Eds.), *Automated Machine Learning: Methods, Systems, Challenges* (pp. 3–33). Springer. https://doi.org/10.1007/978-3-030-05318-5_1

Galke, L., Saleh, Y., Baird, A., & Scherp, A. (2022). *Are we really making much progress in text classification? A comparative review* (arXiv:2204.03954). arXiv. https://arxiv.org/abs/2204.03954

Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L., & Brown, D. E. (2019). Text classification algorithms: A survey. *Information, 10*(4), 150. https://doi.org/10.3390/info10040150

Li, Q., Peng, H., Li, J., Xia, C., Yang, R., Sun, L., Yu, P. S., & He, L. (2022). A survey on text classification: From traditional to deep learning. *ACM Computing Surveys, 54*(3), Article 62. https://doi.org/10.1145/3495162

Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, M., & Gao, J. (2021). Deep learning–based text classification: A comprehensive review. *Journal of Big Data, 8*, 53. https://doi.org/10.1186/s40537-021-00444-8

Setiawan, Y., Iriawan, N., Purnomo, M. H., & Aji, A. P. (2024). The optimization of n-gram feature extraction based on TF-IDF for cyberbullying document classification. *Data Science Journal, 23*, 31.